THIS IS THE TITLE OF YOUR THESIS OR DISSERTATION

WHICH MAY SPAN MULTIPLE LINES

(BUT NO MORE THAN 3 LINES)

by

Chris Jordan Doe

A thesis submitted in partial fulfillment
of the requirements for the degree

of

Master of Science

in

Engineering

MONTANA STATE UNIVERSITY
Bozeman, Montana

May 2025

©COPYRIGHT

by

Chris Jordan Doe

2025

All Rights Reserved

DEDICATION

I dedicate this to all MSU students who use LaTeX. Dedication is optional and may be no longer than one page, single spaced, and should precede the acknowledgments page.

ACKNOWLEDGEMENTS

I would like acknowledge... Acknowledgments must be double spaced and is limited to one page. Consider that you may need to include a funding acknowledgement.

VITA

Chris Jordan Doe was born in Bozeman, MT in 1893. Raised by Champ, Chris is a true Bobcat. Chris attended Bozeman High and graduated with honors. After high school, . . .

If you include a vita, it should contain the full name of the author, date and place of birth, parentage, secondary education, and collegiate degrees. The vita should be written in essay form in the third person and may not exceed one single-spaced page.

TABLE OF CONTENTS

1. INTRODUCTION 1

Machining Overview 1
Lathe 1
Mill 3
CNC Based Manufacturing 3
Observability of Machining Loads 4
Machine Load Transmission 6
Characterizing Load at the Cutting Edge 8
Electrical Overview 10
DC Motor 11
Brushless DC Motors 11
Hybrid Stepper Motors 12
Field Oriented Control 13
Control Systems Overview 14
Proportional Integral Derivative Control 15
Active Disturbance Rejection Control 18
Hardware Platform Overview 19

2. FIELD ORIENTED CONTROL WITH A TWO-PHASE HYBRID STEPPER MOTOR 24

Prior Work and Contributions 24
System Architecture 26
Testbed 27
Machining Application Constraints 28
Motor Parameters and Identification 29
Parameter Sources 29
Model Parameter Integration 30
Torque Constant 30
Winding Resistance and Inductance 32
Rotor Inertia 33
Viscous Damping 33
Ball Screw and Nut Inertia 34
Total Inertia 34
Stepper Motor Model 35
Electrical Model 35
Drive Stage Voltage 35
Phase Winding Model 39
Electromechanical Model 40

TABLE OF CONTENTS – CONTINUED

Phase Current Integration 41
Flux Linkage and Torque Equation 42
Mechanical Model 44
Electromagnetic Torque 44
Rotor Dynamics 46
Rotor Angle 49
PCB Model 50
Encoder Emulation 51
Current Sensing 55
Load Cell Emulation 59
Control System 60
Control Architecture 60
Feedback Signal Processing 62
Current Decoding 62
Encoder Decoding 63
Velocity Estimation 65
Rotor-Frame Currents 67
Plant Models for Control Design 67
Velocity Loop Input Gain 68
Current Loop Input Gain 69
Discrete-Time First-Order ADRC 69
Extended State Observer and Control Law 70
Minimum-Footprint Implementation 71
Closed-Loop Properties 73
Position Loop 74
Velocity Loop 75
Current Loops 77
Voltage Output and PWM Generation 77
Fixed-Point Implementation 80
Simulation Experiments 81
Experimental Setup 81
Tracking Performance 82
Load Estimation from the Current Reference 83
Disturbance Rejection 84

3. TITLE OF MANUSCRIPT YOU ARE INCLUDING AS A CHAPTER 86

Contribution of Authors and Co-Authors 86
Manuscript Information 87

TABLE OF CONTENTS – CONTINUED

4. CONCLUSION 88

REFERENCES CITED 90

APPENDIX: Example Code 94

LIST OF TABLES

Table Page

1. Table 1 Control loop update rates of the Raspberry Pi 4 and FPGA implementations. 23
2. Table 2 Resource utilization for the first order ADRC [8] 25
3. Table 3 Electrical and mechanical parameters of the Dings' Motion NEMA 23 stepper motor, model 23H2065. [18] 30
4. Table 4 Measured versus datasheet stepper motor inductance (at 1 kHz) 32
5. Table 5 Plant parameters used in the simulation and source of parameter. 35
6. Table 6 Input signals of the voltage synthesis stage. 39
7. Table 7 Parameters of the electrical model. 44
8. Table 8 Signals of the mechanical model. 50
9. Table 9 Parameters of the mechanical model. 50
10. Table 10 Quadrature states generated from the encoder count. 54
11. Table 11 Signals in the encoder subsystem. 55
12. Table 12 Parameters of the encoder emulation. 55
13. Table 13 Signals of the current sensing emulation. 59
14. Table 14 Parameters of the current sensing emulation. 59
15. Table 15 Closed-loop bandwidth ω_{CL} and observer bandwidth ω_{ESO} of the control loops. 62
16. Table 16 ADRC design parameters and coefficients, computed with $T_\omega = 400\,\mu s$ (velocity) and $T_i = 20\,\mu s$ (current). 78
17. Table 17 Fixed-point data types of the controller. 80
18. Table 18 Simulation test cases. 82
19. Table 19 Tracking performance of the unloaded axis. 83
20. Table 20 Measured and predicted slope ratios for each spring. 85

LIST OF FIGURES

Figure			Page
1.	Figure 1	Cutting force components in turning	2
2.	Figure 2	Load path from cutting force to motor current	6
3.	Figure 3	Cutting force in turning and milling	9
4.	Figure 4	Brushed DC motor construction	11
5.	Figure 5	Brushless DC motor construction	12
6.	Figure 6	Hybrid stepper motor construction	13
7.	Figure 7	Clarke and Park transforms for a three-phase motor	14
8.	Figure 8	Park transform for a two-phase stepper motor	15
9.	Figure 9	PID controller	16
10.	Figure 10	Cascaded position and velocity control	17
11.	Figure 11	Preliminary PID current control	18
12.	Figure 12	First-order ADRC structure	19
13.	Figure 13	FPGA architecture	20
14.	Figure 14	Partition of the Simulink model	22
15.	Figure 15	Overview of the simulated system	27
16.	Figure 16	Top level of the Simulink model	28
17.	Figure 17	Mechanical test bench	29
18.	Figure 18	Sensitivity of the step response to load inertia	31
19.	Figure 19	Simulink model of the H-Bridge drive stage	36
20.	Figure 20	The StepperElectrical subsystem	36
21.	Figure 21	Mechanical model of the stepper motor	45
22.	Figure 22	Full-step validation of the motor model	46
23.	Figure 23	The StepperMechanical subsystem	47
24.	Figure 24	The rotor angle subsystem	49

LIST OF FIGURES – CONTINUED

Figure		Page
25. Figure 25	The encoder emulation subsystem	52
26. Figure 26	Quadrature encoder waveforms	52
27. Figure 27	Encoder channel generation logic	53
28. Figure 28	The ADC Emulator subsystem	57
29. Figure 29	Top level of the FOC controller	61
30. Figure 30	The Encoder Decoder subsystem	64
31. Figure 31	Period-based velocity estimate	67
32. Figure 32	The `MFP_ADRC_1st` subsystem	73
33. Figure 33	Velocity-loop tuning map	76
34. Figure 34	Original and selected velocity-loop tunings	76
35. Figure 35	The PWM generator subsystem	79
36. Figure 36	Unloaded step response	82
37. Figure 37	Current reference and load cell force	84
38. Figure 38	Estimated versus measured force	84
39. Figure 39	Response to a torque impulse	85

NOMENCLATURE

μ Dynamic viscosity
$\mathbf{n}$ Normal vector
$\mathbf{u}$ Velocity vector

ABSTRACT

The abstract must be single spaced and no more than 350 words. The abstract must contain the following elements: (1) statement of the problem, (2) procedure or methods, (3) results, and (4) conclusions. Mathematical formulas, abbreviations, diagrams, and other illustrative materials should not be included. It should be written to be understood by a person who does not have expertise in the field.

1

INTRODUCTION

Modern manufacturing relies heavily on producing precise, intricate metal components. This is often achieved through subtractive manufacturing techniques using equipment such as milling machines and lathes. These machines have been in use for approximately two hundred years [21, 26], and during this time have been continually improved. Manual control of these machines has been incrementally replaced by increasingly capable computerization [21, 31]. Modern milling machines and lathes often use Computer Numerical Control (CNC), which removes the need for a human machinist to control each detail of the machining process through manual inputs [21]. Due to this continued computerization, these machines have grown in complexity, enabling more advanced machining operations, including advanced work with ductile materials, brittle materials, and increasingly complex part geometry [26, 33]. Continued development has also introduced new systems to these machines, including systems that allow the Machine to autonomously detect in real time a broken tool and replace the tool, detect abnormally high stress loads, or monitor motor loads across the machine. These innovations are significant because they enable the machines to autonomously catch in real time errors that previously would have been caught by a human, and with CNC machines could have propagated into damaged parts or damaged machines. These innovations work to reduce scrap part rates, damaged tooling, and critically machine downtime [27, 39].

Machining Overview

Lathe

A lathe performs subtractive manufacturing by rotating the workpiece around the spindle axis, designated as the Z axis, while translating the cutting tool in the X and Z

axes. The X axis is commonly used to move the tool radially and controls the finished diameter of the workpiece. The Z axis moves along the length of the workpiece and is used to control the axial extent of the cut. Given the workpiece rotates while the tool traverses a plane containing the axis of rotation, all features are produced as surfaces of revolution that are axially symmetric about the Z axis [1]. Lathes are commonly used to achieve part geometries that require turning outer diameters, boring internal diameters, facing, and drilling and tapping holes along the Z axis. Contoured profiles, including hemispherical and other curved forms are produced by coordinating the X and Z motions simultaneously [21]. A conventional lathe has no Y axis; the Y axis is traditionally used only on mill-turn machines and multi-axis turning centers. The Y axis enables the off axis of rotation features which would break axial symmetry [26].

During turning passes, the tool remains engaged with the workpiece continuously, and in the absence of material variation or progressive tool wear, the cutting force is approximately constant. This force is conventionally resolved into three orthogonal components, tangential, radial, and feed [1]. The magnitude of each of these forces is determined largely by the tool geometry, machine geometry, and the part characteristics. Large contributions to this are determined by the tool geometry and the lead and rake angles [1]. The three components are illustrated in Figure 1.

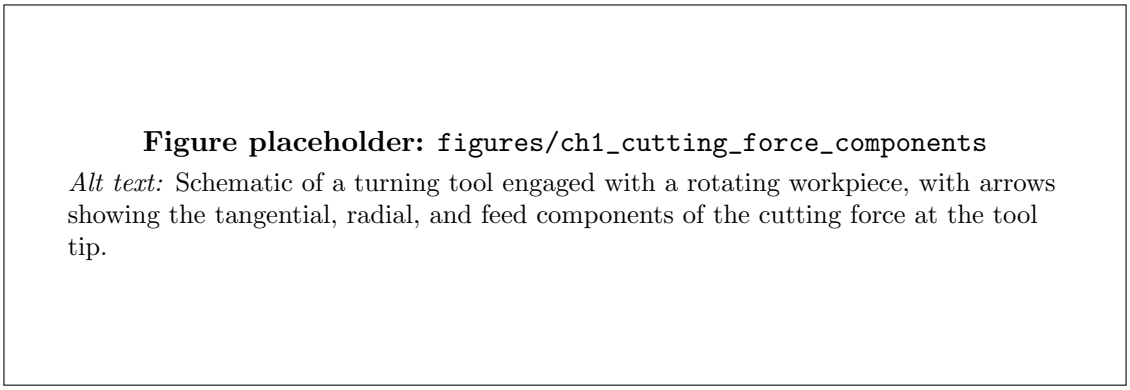


Figure 1: Cutting force at the tool tip resolved into tangential, radial, and feed components.

Mill

A milling machine performs subtractive manufacturing by rotating the cutting tool about the spindle axis, designated the Z axis in a milling machine. Part geometry is achieved by moving the tool and workpiece relative to each other along three orthogonal directions. On a standard vertical machining center, the table carries the workpiece in the X and Y axes while the spindle head provides motion in the Z axis setting the depth of cut. Through these three degrees of freedom, material can be removed at arbitrary locations on the workpiece, so a far wider range of geometry can be produced compared to a lathe. Additional rotary axes extend this part geometry further. Four-axis machines add a single rotary axis about the X or Y axis and five-axis machines add a second rotary axis allowing the tool to approach the workpiece at a continuously variable orientation and make undercut and complex contoured surfaces without rotational symmetry possible [1]. The cutting force in a milling operation is applied to the workpiece through the flutes of the rotating tool, and each flute enters and leaves the cut once per spindle rotation. Engagement therefore is intermittent rather than continuous, and the load present on the machine is periodic at the tooth-passing frequency. The tooth-passing frequency is equal to the spindle rotational speed multiplied by the number of flutes on the cutting tool [1, 21, 26].

CNC Based Manufacturing

Both lathes and mills can be improved by the addition of computer based control, and machines of this type are known as Computer Numerical Control (CNC) lathes and mills. Through CNC, tool paths are specified as coordinated motion commands rather than performed by an operator adjusting hand-wheels and cranks. CNC toolpaths allow for simultaneous coordinated multi-axis motions that are required for complex contoured geometry that would be impractical to produce manually [1, 21]. On servo based machines, motion is regulated by nested closed-loop position and velocity controllers acting in each

axis [1]. The controller compares the commanded position against measured position at each update of the servo loop, which often occurs faster than the machine could reach the desired location, position error is corrected continuously. This control system allows CNC machines to hold a high level of precision and to reproduce the same result across many parts. Machines driven by stepper motors are commonly used and typically operated in an open-loop configuration. This principle takes advantage of the internal stepping motion a stepper motor provides to achieve reproducible and precise control in a simplified method compared to servo based machines, trading some precision for lower cost [8, 22, 32].

Adding sensing beyond the position control loop permits tighter control and the recording of process data during machining. This data can be retained on a per-part and per-machine basis and support process improvements across a production run in a manner that manual machining did not allow for [1, 14, 34, 39]. High-performance CNC machines are typically expensive and occupy large floor footprints. These large often cast based footprints are used to raise the mass and structural damping of the machine, attenuating vibration and disturbance forces before tool disturbance can occur. This contributes substantially to the precision and repeatability these machines can achieve [1]. If these disturbances can instead be characterized and rejected within the control system, a portion of the precision obtained by this structural mass could in principle be obtained from control, allowing smaller and lighter machines to approach the performance of larger cast based machines. Machines of this type would be usable in applications where the machine must be enclosed and where cost or space required for this enclosure currently constrains what can be produced.

Observability of Machining Loads

Some modern CNC machines are capable of recording substantial data during operation, and that data is presently used in three primary ways: to improve part quality across a production run, to reduce machine cycle time, and to indicate when maintenance should

be performed [34, 35]. Each of these acts after the fact, as the part being cut when the data was collected is already complete. With additional sensing and characterization of the machine, the same information could support correction during the current cut, acting on the workpiece currently being machined rather than the next one. Corrections of this kind would also accumulate, producing a record of which in-process adjustments succeeded which could be applied to a new part geometry without repeating the same trial cuts.

Systems performing monitoring of this kind are already commercially available. The Tool Monitoring Adaptive Control (TMAC), developed by Caron Engineering interfaces directly with a range of CNC controls, including Okuma platforms. The system monitors spindle motor power during the cut and operates on the principle that the power required to cut increases as the cutting edge wears. It applies this logic in two ways, it retires a worn tool or halts the machine on breakage, and in adaptive control mode, it overrides the programmed feed rate to hold spindle motor power near a set limit. Data from these cuts can be retained for later analysis and process improvements [11, 14, 15, 30].

The adaptive capability therefore exists, but it acts on an aggregate quantity. Spindle motor power represents the total power drawn by the cut. It does not resolve the force at the cutting edge into its principal components, and holding that power near a limit regulates the process only in the coarsest sense, without reference to the cutting conditions that have produced these changes. Civello [14] found that cutting force at the cutting edge could not be resolved from the data accurately enough to infer the resulting change in part geometry or surface finish. This limitation was attributed to the sample rate of the TMAC system, where the noise was no longer distinguishable from the signal. The signal was sampled at a rate of 200 Hz, which was too low to resolve the cutting-force signal above the noise. This resulted in the data that showed the process was feasible, however the electrical sampling system would need significant changes to be practical. A system with direct access to the motor's electrical quantities and closer integration with the motor may resolve the signal-to-noise

ratio problem and provide more insight than the TMAC system could.

Machine Load Transmission

When an axis is driven by a ballscrew, the cutting force acting on the carriage produces an axial load on the ballscrew. Given that a ballscrew is backdrivable the axial load appears at the motor as a rotational torque. The magnitude of this torque is set by the screw lead and the efficiency of the drive. The cutting load therefore reaches the motor through the existing drive system, without additional instrumentation added to the machine. Many CNC

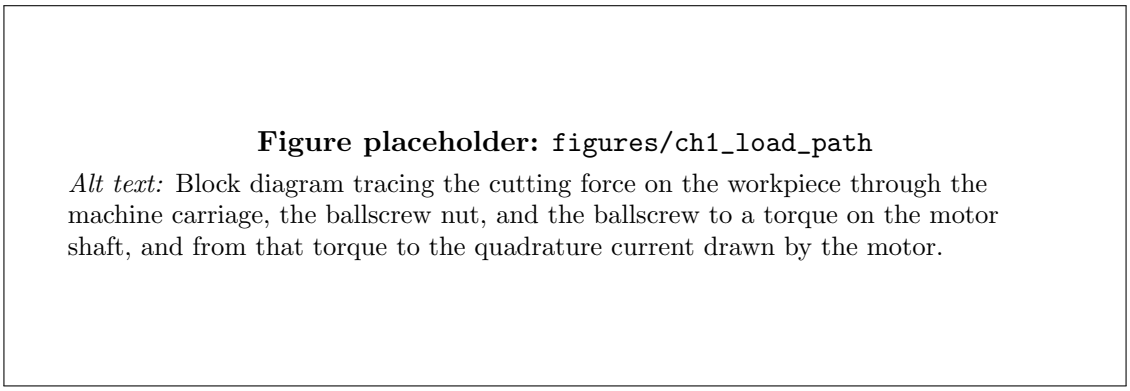


Figure 2: Load path from the cutting force at the tool to the quadrature current of the axis motor.

machines come standard with ballscrews, however those that use alternative drive types such as lead screws can be retrofitted with ballscrews to achieve this behavior and enable this torque transmission. The resulting load path is shown in Figure 2.

Electric motors produce torque as a function of supplied current, the form of this current torque relationship varies on motor design. However, all electric motors fundamentally convert electrical current into a magnetic field that is used to produce torque. Some motor types produce torque at a fixed linear relationship to current, while others follow through a non-linear relationship dependent on rotor angle. Determining the load from the current therefore requires a priori knowledge of the electrical characteristics of the motor [8, 16, 22].

This torque to current relationship motivates the review of motor types in the following section.

Where the relationship is known, the cutting load appears at the axis drive as a disturbance torque, and the controller responds by changing the command current in order to preserve the axis trajectory. The cutting load is consequently present in the current signal and can be recovered from this disturbance. Recovery of this cutting load requires separating this load from the other torques the motor supplies during the same motion. These additional disturbance torques include friction in the system, inertial torque required to accelerate and decelerate the axis, and on a vertical axis the effects of gravity on the load of the carriage. Inertial and gravitational contributions follow from the axis mass and command trajectory can be predicted directly and therefore identified as a known disturbance. Friction is less tractable, comprising Coulomb, viscous, and velocity dependent components that vary with temperature and exhibit a hysteresis at velocity change [1, 4]. These disturbances are therefore estimated rather than predicted, and corrected for in situ through observer-based and signal processing techniques present in the proposed control system [24].

The underlying principle for torque based disturbance characterization is already applied in a limited form on modern CNC machines. This is used where drive current detects events without any added sensor. These methods are commonly described as “sensorless”. The term originates in motor control, where it denotes the estimation of rotor position without a position sensor [13, 16]. In this context it is used in the broader sense adopted in machine tool practice, meaning that the quantity of interest is inferred from signals already present in the drive rather than measured by a sensor added for that purpose. Homing against a hard stop is one example. When the axis reaches the stop, the motion ceases while the controller continues to the command position. This position error accumulates and the commanded current rises toward the motor limit. This sustained rise identifies a hard stop and can be used to establish a machine reference point without a limit switch [3, 38]. This

hard stop referencing method is only practical due to the relatively high speeds at which the controller is able to sample error and detect a disturbance so no damage to the machine occurs during this brief period of time when the hard stop is in contact with the carriage on that axis.

This principle can be expanded to detect other machine changes, tool breakage is another common use case for detecting sudden current load changes. This is normally performed by monitoring the spindle drive rather than an axis drive. The sudden loss of a cutting edge produces an abrupt decrease in spindle load, most commonly a decrease in power as the tool is no longer in contact with the workpiece [13, 39]. This signal is drawn from a different motor, however the principle is the same in both use cases. Both the axial hard stop and the tool breakage are determined by sudden change in current draw in the motor controlling that motion. The present work extends this same principle from an event detection approach to a continuous estimation of the load magnitude.

Characterizing Load at the Cutting Edge

With a motor and control system of sufficient electrical and control bandwidth, current can be sampled rapidly enough to follow variation in the cutting load rather than only tracking an average value. Turning is the appropriate operation in which to establish this relationship first. The tool remains engaged continuously through a turning pass, and during a hemispherical operation translates in both the X and Z axes causing a change in the force vector direction. With these passes, in the absence of material variation or progressive tool wear, the cutting force is approximately steady and the motor torque is likewise steady. Milling by contrast, presents an intermittent load at the tooth-passing frequency [1], superimposing a large periodic component on the signal of interest. Beginning with turning isolates the current-to-force relationship from the periodic relationship which allows it to be established under the simplest available loading condition. The two loading

conditions are compared in Figure 3.

Figure placeholder: `figures/ch1_turning_vs_milling_force`

Alt text: Two plots of cutting force versus time: a nearly constant force for a turning pass, and a force for a milling pass that rises and falls once per tooth, repeating at the tooth-passing frequency.

Figure 3: Cutting force during turning, which is approximately constant, and during milling, which is periodic at the tooth-passing frequency.

Direct instrumentation of the cutting zone is not viable in the target applications, where the machining environment is enclosed and access is limited. Introducing sensors into this environment introduces a non-trivial cost aspect and would expose sensors to potential damage due to conditions internal to the machine during operation[39]. Using already existing mechanics to infer the load from the motor current instead, locates the measurement at the drive, where the necessary electrical quantities are already present in the controller, and requires no instrumentation in the cutting zone at all. Establishing this relationship also opens a path to a quantity that is not otherwise observable during machining. Force at the cutting edge is distributed over the contact between the tool and the workpiece, and the pressure in that contact, rather than the total force, is what governs the material's response to the cut. Pressure follows from force and the area over which it acts, and that area is determined by the chip cross-section and geometry of the cutting tool [1, 14].

Specific cutting energy provides the standard basis for relating the measured force to the condition in the cutting zone [1, 14, 21]. Where force can be recovered from drive current, tool geometry, and chip cross-section from machining parameters, an estimate of pressure at

the cutting edge can be determined without additional measurement.

A pressure estimate obtained through this could be of use where a material's machining envelope depends on the loading conditions it experiences rather than on nominal properties alone. Certain material classes undergo solid-state phase transforms within the range of stress and temperature encountered at the cutting edge, and for these deformations the behavior is dependent on the pressure and temperature history the material has experienced [40]. Machining such materials predictably requires a priori knowledge of the load actually being applied at the cutting edge, which standard commercial CNC control systems do not provide [35, 40].

Recovering pressure in this manner is beyond the scope of the present work, which establishes a relationship between motor current and axis load that any such estimate would depend upon. This approach is identified here because it defines the direction in which this capability could extend, and because the requirements this imposes depend on the method of data collection in the present work. These requirements are bandwidth sufficient to follow the cut, a torque-to-current relationship, and a control system capable of accepting current feedback. Meeting these requirements on a stepper-driven axis requires a motor model, a control architecture capable of maintaining the torque-to-current relationship under load, and a processing platform capable of operating at the required rate. Each of these is developed in the present work.

Electrical Overview

To accurately capture the present work, the fundamentals of common motor types are reviewed. Motor types determine the torque-to-current relationship. Direct Current (DC) motors have a relationship such that the torque produced is directly proportional to the current passing through the motor windings. The torque to current relationship in a DC motor does not depend on the motor rotor angle for torque generation and is

constant regardless of the current rotor angle. Brushless Direct Current (BLDC) motors and Hybrid Stepper Motors do depend on the rotor angle as part of the current to torque output relationship along with the current moving through the motor windings [8, 28].

DC Motor

DC Motors have the most direct relationship for current and torque. In a DC motor, a voltage is applied to the motor windings, which are located on the rotor, and permanent magnets inside on the stator respond to the generated magnetic field and rotation is generated. While this motor type provides a simple installation and simple operation, the mechanical link of using brushes to pass the current from the fixed motor connections through the commutator to the rotating rotor is susceptible to failure from wear and can lead to a shortened motor life compared to technologies such as BLDC motors [8, 28]. The construction is shown in Figure 4.

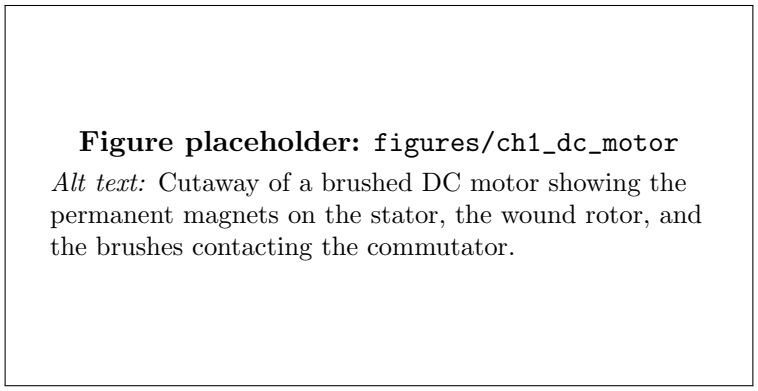


Figure 4: Internal construction of a brushed DC motor.

Brushless DC Motors

Brushless DC Motors are often used in servo-based CNC machines where the longevity and precision control of the axes are required. In these applications a BLDC motor is selected due to the lack of brushes internal to the motor that can degrade with use. With no direct contact between the rotor and the stator, BLDC motors are not susceptible to the same

wear and degradation as a DC motor with brushes in the commutator failing. Therefore BLDC motors have less wear internally that cause premature motor failure. These motors require additional control to operate as these motors require timing to three separate internal phases to properly generate rotational motion. BLDC motors differ from Permanent Magnet Synchronous Motors (PMSM) in the way that the motors are driven and the torque output relationship. In PMSM motors the coil windings are such that an AC signal is required to drive the motor, meaning external sinusoidal control of the voltage is needed. Whereas with the BLDC motor, the motion can be generated with a trapezoidal method instead of a sinusoidal method [8, 28]. The construction is shown in Figure 5.

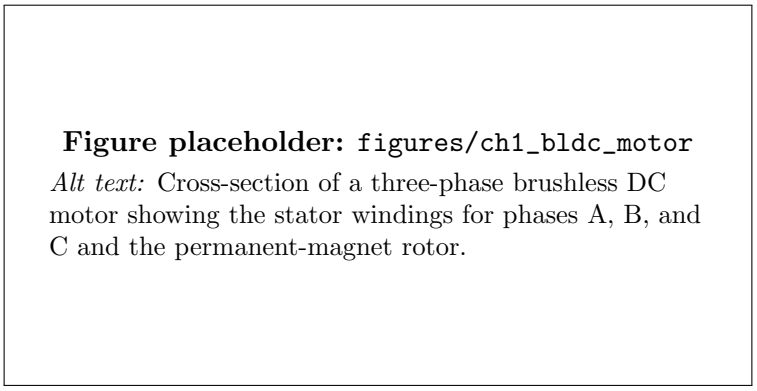


Figure 5: Internal construction of a three-phase brushless DC motor.

Hybrid Stepper Motors

Hybrid Stepper Motors are used when repeatable motion is desired at a low cost. The hybrid stepper motor takes advantage of internal fixed coils that cause predictable and repeatable rotation in a step motion. The internal rotor can be thought of as moving akin to a gear where each coil will advance the gear by one quarter of a tooth. While this is not the direct method used in hybrid stepper motors, this model provides a more clear understanding of the intended use case for the stepper motor. Internal to a stepper motor the rotor and stator poles are set such that both magnetically align at each step at the

maximum magnetic field strength. This electrical and magnetic coupling is what provides the strong holding force of a stepper motor along with the stepping action [8, 16, 22]. A standard method to driving a stepper motor is to use pulse trains alternating between the two phases of the motor. This electrical signal is simple to produce and causes rotational motion of a stepper motor in a way that can be easily discretized [25, 32]. The construction is shown in Figure 6.

Figure placeholder:
`figures/ch1_hybrid_stepper_motor`

Alt text: Cross-section of a two-phase hybrid stepper motor showing the toothed stator poles carrying the phase A and phase B windings and the toothed permanent-magnet rotor.

Figure 6: Internal construction of a two-phase hybrid stepper motor.

Field Oriented Control

Field Oriented Control (FOC) is commonly used on BLDC motors and three-phase power systems to transform the multiple phases into two orthogonal components, the quadrature (q) axis and the direct (d) axis. The quadrature axis is the torque-generating axis that is always leading the electrical angle by 90 degrees while the direct axis is aligned with the rotor flux and therefore is not producing a rotational torque. This method is commonly used on BLDC motors as FOC reduces the complexity of driving the BLDC motor down to two vectors, the Quadrature which is scaled by the required torque output and the Direct which is minimized. Through this transform, the motors are able to maximize the torque produced and linearize the torque-to-current relationship [6, 8, 28]. The transforms are illustrated in Figure 7.

Figure placeholder:
`figures/ch1_clarke_park_three_phase`

Alt text: Vector diagram showing the three stationary phase axes a, b, and c at 120 degrees, the stationary alpha-beta frame produced by the Clarke transform, and the d-q frame rotating at the electrical angle theta e produced by the Park transform.

Figure 7: Clarke and Park transforms for a three-phase motor, from the stationary a–b–c phase axes to the rotating d–q frame.

The present work seeks to show that this relationship can be extended from the BLDC motor to the Hybrid Stepper Motor [7, 16, 25, 32]. The same fundamental property is being used to convert the discrete steps internal to the stepper motor into a continuously rotating system. The Quadrature current is the current solely responsible for generating rotational torque in the stepper motor and the direct-axis current is wasted energy that does not contribute to the rotation or stepping motion of the motor. This is because the direct-axis current generates a flux along the rotor magnet axis which does not generate torque but still passes current through the motor. Through this transform, the Hybrid Stepper Motor will be treated as a synchronous machine where two continuous sine waves can be applied to the two phases of the stepper motor at 90 degrees out of phase to generate rotation. The phase of both sine waves is directly linked to the motor rotor electrical angle keeping both in phase with the motor to generate the rotation [7, 16, 25]. Figure 8 shows the two-phase case.

Control Systems Overview

The current work covers two control systems which necessitates a review of the basics of both control systems. Traditional control systems are computed and designed in the frequency domain while modern control systems are often completed in the time domain

Figure placeholder: `figures/ch1_park_two_phase`

Alt text: Vector diagram of a two-phase hybrid stepper motor showing the orthogonal phase A and phase B axes, and the d and q axes rotating at the electrical angle theta e, with the current vector aligned to the q axis.

Figure 8: Park transform for a two-phase hybrid stepper motor. The phase axes are already orthogonal, so no Clarke transform is needed.

through the use of linear algebra [5, 8]. The initial design for implementing FOC control on a stepper motor was achieved through a cascaded PID control loop. This provided some initial results and proved the system was worth pursuing further. One of the limitations of this solution came from the hardware running it, a Raspberry Pi 4. This introduced inconsistencies in timing and created a non-deterministic control system. Due to these computational limits, the system was moved to an FPGA which also provided the possibility of doing computation using matrices while having deterministic timing. With this, the present work uses an Active Disturbance Rejection Control method to remove additional disturbances in the system through the use of observers. Despite the advantage of having matrices to represent the control law of the system, a minimum footprint Active Disturbance Rejection Control algorithm was used. This converted the matrix mathematics to a transfer function that requires fewer FPGA resources [8, 23].

Proportional Integral Derivative Control

One classical approach to control systems for CNC machines is to use Proportional Integral Derivative (PID) control to control the motion of the CNC machine. A PID system makes use of comparing the feedback to the target reference point to calculate the error. This traditional control system is often implemented as a single input single output system

(SISO) where the position and velocity control are handled separately[1, 8].

The PID control law is

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}, \qquad e(t) = r(t) - y(t), \tag{1}$$

where r is the reference, y is the measured output, and K_p, K_i, and K_d are the proportional, integral, and derivative gains [5]. The structure of the controller is shown in Figure 9.

Figure placeholder: `figures/ch1_pid_block`

Alt text: Block diagram of a PID controller: the error between the reference and the measured output feeds parallel proportional, integral, and derivative paths whose sum is the control signal applied to the plant.

Figure 9: Parallel PID controller acting on the error between the reference and the measured output, Equation (1).

Through this a position motion profile can be created where the velocity control system is quickly saturated to the reference feedrate specified in the G-Code for the program. Through this, a responsive system that is able to reach the target position which a high level of accuracy is achieved while maintaining the desired feedrate on the machine. This modification of a traditional position control system allows for a set velocity target to be maintained during cutting operations which is vital to the proper functionality of a milling machine or lathe. Constant cutting velocity provides a consistent chip load which produces a more precise final result [1].

To achieve FOC control using PID systems, a preliminary design was created that utilized a PID controller for the position, velocity, quadrature current, and direct current. This system involved cascading the control system such that the output of one controller

Figure placeholder: `figures/ch1_cascaded_position_velocity`

Alt text: Block diagram of cascaded position and velocity loops: the position controller output is the velocity reference, the velocity controller output is the current or torque command, and position and velocity feedback return from the motor encoder.

Figure 10: Cascaded position and velocity control, in which the position controller supplies the velocity reference.

became the reference for the next controller. Through this, a reference quadrature current would be established that would provide the correct current to generate the target velocity and move the motor to the desired location at the desired velocity. By using the Park Transform, the quadrature current was made linear in respect to the output torque [6, 16]. The quadrature current reference is treated as a step target with a constant current being related to a constant acceleration while under no-load operation. During a constant load, instead of the quadrature current settling at zero with no motion, the quadrature current will settle at a value directly proportional to the load on the system. The current loops are shown in Figure 11. Running in parallel to the quadrature current control loop was the direct current control loop with a constant reference input of zero. Through this, a linear current target change can be attributed to disturbances in the motion of the motor such as an increase of torque working against the rotation of the stepper motor.

Early success was found with this system, however this was implemented on a Raspberry Pi 4 B 8 Gb model which introduced limitations in how fast the control system could update and how fast feedback could be collected. This limited the control system to updating at approximately 200 Hz. At 200 Hz the update period is 5 ms, which is longer than the 3 ms electrical time constant of the motor windings. This limitation meant the time constant

Figure placeholder: `figures/ch1_pid_current_loops`

Alt text: Block diagram of the preliminary PID current control: measured phase currents pass through the Park transform to give i d and i q, two PID controllers regulate i d to zero and i q to its reference, and the inverse Park transform produces the phase voltage commands.

Figure 11: Preliminary PID current control in the rotor-fixed d–q frame.

of the stepper motor was shorter than the update period of the control system. Through this, no meaningful change in current could be detected that could be related to the change in torque observed. However, this initial system did prove that a stepper motor could be run using FOC and that a change in quadrature current was noticeable in the offset of the waveform as the load increased. Given limitations in the software system on the Raspberry Pi and potential additional disturbances in the mechanical system imperfections, a conclusive relationship between torque and current could not be determined. However, the initial data showed an increased trend in quadrature current when torque increased.

Active Disturbance Rejection Control

A principal limitation of PID control is the delay in detecting disturbances through the error signal from the plant. With PID control, there is no predictive work being done to estimate the error that will result from the plant and how to separate out error from the plant versus error that is the result of a disturbance. With the Active Disturbance Rejection Control (ADRC) the error of the plant is predicted through an extended state observer (ESO). Modeling this error using the current reading from the plant allows for converging the estimate of the plant response to the control signal. Through this, the way the plant will react to a given control signal can be efficiently modeled and disturbances

to this control signal predicted [24]. By using an ADRC system, the disturbances of the plant can be modeled and removed which allows for a clean control signal to be provided to the plant. This results in a quadrature current reference that is more predictable and will account for external loads that were not modeled in the stepper motor [24]. To achieve this result, a minimum footprint ADRC system was modeled in Simulink where an ADRC is used to control the velocity of the motor and the current in both the quadrature and direct phase. A proportional controller is used to control the position of the motor, as the output of the position controller following convention should saturate to the velocity limit imposed by the machining operation. The structure of the ADRC is shown in Figure 12.

Figure placeholder: `figures/ch1_adrc_block`

Alt text: Block diagram of first-order ADRC: the plant output y feeds an extended state observer that estimates y and the total disturbance f; the control law subtracts the disturbance estimate from a proportional term and divides by b0 to form the control signal u applied to the plant.

Figure 12: First-order ADRC. The extended state observer estimates the output and the total disturbance, and the control law cancels the estimated disturbance (Equations (59) and (61)).

Hardware Platform Overview

A Field Programmable Gate Array (FPGA) is an integrated circuit made up of an array of configurable logic blocks which are connected through a programmable routing network. Each logic block contains lookup tables (LUTs) that can implement combinational logic functions, along with flip-flops that hold the state of the circuit between clock cycles. Many FPGAs also include dedicated resources such as Digital Signal Processing (DSP) slices for fast multiplication and accumulation, block RAM for on-chip memory storage, and configurable

input and output blocks for interfacing with external hardware. The behavior of the FPGA is described using a Hardware Description Language (HDL), such as VHDL or Verilog. This description is synthesized into a netlist of logic elements, placed and routed onto the physical resources of the device, and then loaded onto the FPGA as a configuration bitstream [8]. A simplified view of this architecture is shown in Figure 13. The key difference between an FPGA and a traditional processor such as the Raspberry Pi 4 is in how an algorithm is executed. A microprocessor executes a sequence of instructions in a serial fashion with dedicated instructions to use, while an FPGA implements the algorithm directly as digital circuitry. Through this, independent operations such as the Clarke and Park transforms, the extended state observers, and the encoder decoding can all be computed at the same time in separate regions of the device rather than competing for time on a single processor.

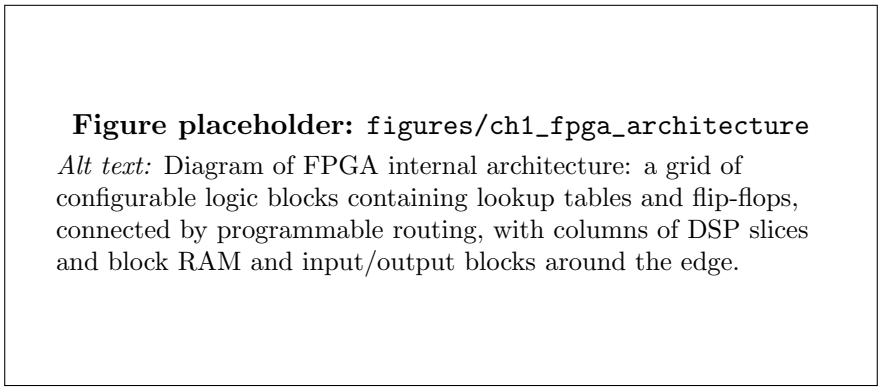


Figure 13: Simplified internal architecture of an FPGA.

A System on Module (SoM) FPGA packages the FPGA together with the supporting hardware required for operation onto a single compact circuit board. This supporting hardware typically includes the power regulation and power sequencing, the clock sources, external DDR memory, and non-volatile flash memory for storing the configuration bitstream. The SoM is mounted onto a carrier board through high-density connectors, where the carrier board provides the application-specific hardware such as the motor drivers, the current sensing circuitry, and the encoder connections. Many SoMs are built around an SoC

FPGA, which combines hard processor cores and programmable logic on the same chip. In these devices, the processing system can run an operating system to handle tasks that are not time critical, such as parsing G-Code, communicating with a user interface, and logging data, while the programmable logic handles the time critical control loops. The two sides of the device communicate through an on-chip bus such as the Advanced eXtensible Interface (AXI) [8]. The advantage of using a SoM is that the most difficult portions of the board design, such as the high-speed memory routing and the power sequencing of the FPGA, have already been designed and validated by the manufacturer. Therefore, the design effort can be focused on the motor control hardware and the control system itself rather than on the supporting electronics of the FPGA.

The control system in the present work was designed in Simulink with the intent of being converted to HDL for synthesis. While Simulink allows a control system to be modeled with very few restrictions, a model intended for HDL synthesis must describe something that can be feasibly built out of physical digital logic, which imposes several constraints on the design. The model must be in discrete time with a fixed-step solver, so continuous blocks such as integrators and transfer functions are replaced with discrete equivalents. The transfer function form of the minimum footprint ADRC is well suited to this, as a discretized transfer function maps directly to a set of delays, multipliers, and adders. The model is also converted to fixed-point arithmetic, as floating-point arithmetic consumes a large amount of FPGA resources, which requires a word length and fraction length to be selected for each signal to balance range, resolution, and resource usage [8]. Only blocks with HDL support may be used, so functions such as the sine and cosine required for the Park transform are implemented using lookup tables. Additionally, long chains of arithmetic operations must be pipelined to meet the timing of the FPGA clock, and the resulting latency must be included in the simulation so that the behavior of the model matches the behavior of the hardware. The resulting partition of the model is shown in Figure 14.

Figure placeholder: `figures/ch1_simulink_partition`

Alt text: Top-level diagram of the Simulink model divided into the HDL-synthesizable controller subsystem and the simulation-only subsystem containing the stepper motor and PCB models, with the PWM, ADC, and encoder signals crossing the boundary.

Figure 14: Partition of the Simulink model into the HDL-synthesizable controller and the simulation-only plant.

The largest benefit of moving the control system to an FPGA is deterministic timing. On a general purpose processor running an operating system, as with the Raspberry Pi 4 used in the initial design, the time between control loop updates varies due to the operating system scheduler, interrupts, other running processes, and memory access delays. A control system designed in discrete time assumes that the sample period is constant, and the integral terms, derivative terms, and extended state observer are all computed using this sample period. When the actual period differs from the assumed period, the integration accumulates error, the derivative estimate becomes noisy, and the observer estimate deviates from the true state of the plant. This timing jitter also acts as a varying delay in the feedback path which reduces the phase margin of the control system [8, 12, 19].

On an FPGA, each control loop update completes in a fixed number of clock cycles derived from the system clock. Through this, the sample period is constant to within the accuracy of the clock, and the latency from a measurement to the corresponding control output is known exactly and can be included in the design of the control system. Because each operation runs in its own dedicated logic, adding computation or an additional motor axis does not affect the timing of the existing control loops. The sampling of the phase currents can also be synchronized with the PWM signals driving the motor, such that the

current is measured at the same point in every PWM cycle. This isolates the sampling from the switching noise of the motor driver [9]. Together, these properties allow the control loop to be updated at a rate many times faster than the approximately 200 Hz achieved on the Raspberry Pi 4, and faster than the electrical time constant of the stepper motor windings. In the present design the current loops update at 50 kHz, 250 times the 200 Hz rate of the Raspberry Pi 4 system. The update rates of both implementations are compared in Table 1

Table 1: Control loop update rates of the Raspberry Pi 4 and FPGA implementations.

Implementation	Function	Update rate	Timing
Raspberry Pi 4	All loops (cascaded PID)	≈ 200 Hz	Varies with operating-system scheduling
FPGA	Current loops	50 kHz	Fixed, 4000 clock cycles
FPGA	Velocity loop	2.5 kHz	Fixed, 80000 clock cycles
FPGA	Feedback processing, PWM	200 MHz	Every clock cycle

FIELD ORIENTED CONTROL WITH A TWO-PHASE HYBRID STEPPER MOTOR

Under FOC, a hybrid stepper motor is treated as a PMSM, and the quadrature current in the motor is directly linear with the load torque with a slope of K_t [7, 16]. Through this relationship, the torque load on the motor can be characterized and estimated. To test this relationship, a Simulink model was constructed using an ADRC based control system and a stepper motor model that matches a NEMA 23 stepper motor. The original model was produced as part of a previous thesis focused on the analysis of the resource utilization of different control systems on an FPGA. The model that was produced for that work did not simulate as the focus of that work was on resource usage analysis [8]. This existing model was purely a mathematical representation of ADRC and stepper motor; no logic was present for the handling of real-world signals that would be required to integrate the stepper motor and supporting electronics with the FPGA. The model did not simulate PWM, ADC codes, the loadcell, and encoder pulses. The model relied on providing analog voltages to the stepper motor and receiving feedback in the form of radians and amps. This feedback is not directly accessible on the designed test bench and therefore had to be modified to be compatible with the PCB and mechanical test bench. Therefore, a PWM generation model, ADC model, load cell logic, and encoder logic was built to replicate the work done on the PCB. This included applying the same filters that are present in analog systems on the PCB to the Simulink model, along with modeling the sensors to provide real-world feedback signals. The goal of this work was to accurately model the complete system under test such that the data collected from the simulation would be reflective of real-world data.

Prior Work and Contributions

The original continuous time, double floating point model was created by Dr. Edens, which was then revised by Caleb Binfet to include solving in the discrete time and partial

work on the fixed point conversion. From here, [8] shows the resource utilization of the MFP First Order ADRC design implemented on the AMD KR260 Robotics Starter Kit SoM in Table 2.

Table 2: Resource utilization for the first order ADRC [8]

Resource	**First Order ADRC**
Multipliers	65
Adders/Subtractors	69
Registers	18
Total 1-bit Registers	240
Multiplexers	79

Source: [8].

The low resource utilization of this design made it an ideal choice to provide the disturbance rejection for parameters of the stepper motor and testing that could not accurately be captured in a simulation. These include friction, bearing imperfections, linear rail imperfections, ballscrew imperfections, and resistive voltage drop in the windings. When this model was transferred to begin work on integration with the PCB and mechanical system, the provided model did not simulate. This led to the work done to fix the model and adapt the model to work with the PCB. The model was rebuilt in the discrete time system to fix integration issues, the fixed point precision was reworked to use standardized values avoiding conflicts in future work, and additional peripheral simulations were implemented. This system represents the most complete model of the stepper motor created in the Systems Engineering in Extreme Environments (SE3) lab and provided a fourth iteration on showing the relationship between the quadrature current and the torque produced. Initial work was based on an Arduino system that provided an initial proof of concept that the stepper motor would rotate with voltage as directed by the FOC algorithm. From here premade 3-phase BLDC FOC compatible motor drivers were tested to seek an off-the-shelf solution to the problem. Ultimately, these boards were not compatible with the stepper motor in the desired configuration and a custom implementation was created on a Raspberry Pi. The Raspberry

Pi system, described in Section Proportional Integral Derivative Control, proved that control algorithm and theory would work end-to-end, however its non-deterministic timing led to difficulties in recreating data between experiments. From this, the FPGA based approach was determined to be an ideal solution and implementation work began. This required reworking the original Simulink approach that provided a cascaded PID system with continuous time and double floating point precision to a discrete time fixed point precision model. This provided an opportune time to implement the ADRC control approach instead of the PID approach.

System Architecture

The Simulink model is split into two primary subsystems, the control system and the stepper motor. The stepper motor model includes the stepper motor and the emulation of load cell, ADCs, and the rotary encoder. This subsystem is not designed to be synthesized into HDL; this is purely constructed for simulation purposes. The control system model is designed to be synthesized into HDL, this subsystem of the simulation contains the ADRC controllers, and the logic to process the raw data from the ADCs, load cell, and encoder. To replicate real signals the model was modified to supply a PWM waveform that can be directly output from the FPGA to the PCB, being compatible with the H-Bridge motor drivers on the PCB. This work moved forward with replicating the physical system in the simulation such that the simulation would produce replicas of the required signals and physical properties of the physical system. The work was then expanded to include modeling of the ballscrew connected to the stepper motor and to model a spring applying a linear load to the ballscrew creating a torque on the stepper motor. Once this was completed, various spring loads were applied to the simulated system allowing for testing the relationship between the quadrature current and the torque of the motor. Figure 15 shows the system at a high level, and Figure 16 shows the top level of the Simulink model.

The PWM is generated as a boolean value that is turned on and off based on a counter running at 200 MHz, which provides a resolution of 0.012 V with a carrier frequency of 50 kHz, the direction is set through a second boolean. These two outputs per phase allow for driving a standard H-Bridge motor controller to control each phase of the stepper motor independently.

The stepper motor model provides as feedback, three 16-bit ADC codes, booleans for the rotary encoder A and B phase pulse and a boolean for the Z index of the rotary encoder. The current in each phase is reported on two of the three ADCs, and the third provides the voltage from the load cell.

During debugging of the model, the stepper motor model was setup to provide the simulated angular position of the rotor, angular velocity, and electrical angle. These values were used to confirm the simulation was properly processing the values through the emulated sensors and reconstruction. These values are unused in the control system.

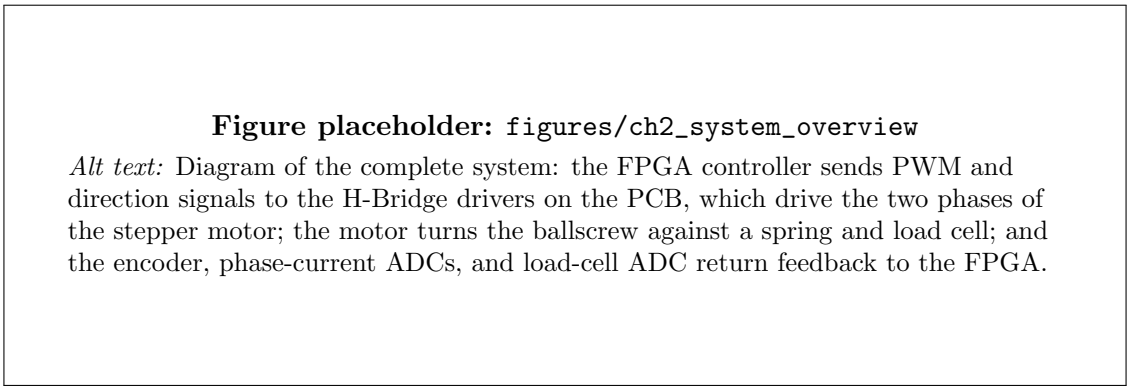


Figure 15: High-level view of the simulated system: FPGA controller, PCB, stepper motor, and ball-screw test bench.

Testbed

To test the performance of the stepper motor with FOC control, a mechanical test bench was constructed that provides dimensional constraints that match the X-Axis of the Mini-Tech CNC mill. This was selected to demonstrate the FOC algorithm would work over

Figure placeholder: `figures/ch2_simulink_top_level`

Alt text: Screenshot of the top level of the Simulink model showing the Stepper Motor FOC Controller subsystem connected to the PCB and physical system model subsystem through the PWM, ADC, and encoder signals.

Figure 16: Top level of the Simulink model.

a full length of displacement for a mill compared to the 152 mm (six inches) of displacement present on the Raspberry Pi based test bench. This new test bench provides a system designed to isolate some of the friction issues present on the previous system and provide a repeatable method to test different load conditions of the motor. The mechanical test bench features a BIF1605-5RRG0 THK ballscrew and caged nut, and two Transducer Techniques MLP-50-C0 load cells mounted in plane with the ballscrew to minimize the rotation caused by a spring on a single side of the ballscrew. The ballscrew has a 5 mm lead, approximately 90 % efficiency, with an inertia of $2.84 \times 10^{-5}\,\mathrm{kg\,m^2}$ with the caged nut adding an inertia of $3.61 \times 10^{-7}\,\mathrm{kg\,m^2}$. These parameters were provided by the manufacturer of these parts, THK [37]. These parameters were then modeled in the simulation. In the simulation the two MLP-50-C0 load cells are modeled as a single load cell that fully captures the load. The test bench is shown in Figure 17.

Machining Application Constraints

With the future goal of applying this work to CNC machining applications, additional constraints are imposed with the goal of maintaining parameters that are indicative of real world machining loads. As such, the velocity is limited to $35.5\,\mathrm{rad\,s^{-1}}$, which in this model corresponds to approximately 339 rpm, which is $28\,\mathrm{mm\,s^{-1}}$ (66 in/min). In a machining

Figure placeholder: `figures/ch2_testbench_cad`

Alt text: CAD model of the mechanical test bench with labeled parts: the NEMA 23 stepper motor and encoder, the THK BIF1605 ballscrew and caged nut, the carriage, the load spring, and the two MLP-50 load cells mounted in plane with the ballscrew.

Figure 17: CAD model of the mechanical test bench.

application this is a high feedrate for machining 6061 aluminum [29]. Through this limit, the relationship between torque and current can be determined. This limit, provides a starting point for future work that implements FOC control in machining applications. With the imposed constraint of a NEMA 23 stepper motor to match the existing Mini-Tech mill, the current limit and voltage limit were imposed of $3\,\mathrm{A_{rms}}$ maximum continuous load and $48\,\mathrm{V}$ limit. These constraints were modeled in the simulation to model an accurate system.

Motor Parameters and Identification

This section identifies the plant parameters used in the simulation. The electrical and mechanical parameters of the motor are taken from the manufacturer datasheet, the load-side inertia is computed from the ballscrew catalog, and the phase inductance and back-EMF constant are checked against measurements of the physical motor.

Parameter Sources

The Dings' Motion NEMA 23 Stepper Motor 23H2065 was selected for testing due to the electrical parameters of this motor being similar to the Mini-Tech CNC Mill. This mill has been used for precision machining work within the SE3 lab and provides a clear baseline for comparing future work implemented on a mill. The reported electrical characteristics of

the Dings' Motion 23H2065 motor are listed in Table 3.

Table 3: Electrical and mechanical parameters of the Dings' Motion NEMA 23 stepper motor, model 23H2065. [18]

Parameter	Symbol	Value	Tolerance / Condition
Rated current (per phase)	I_{rated}	3.0 A	RMS
Phase resistance	R	0.9 Ω	±10% at 20 °C
Phase inductance	L	2.7 mH	±20% at 1 kHz
Holding torque	T_{h}	1.60 N m	—
Rotor inertia	J_{r}	350 g cm^2	—
Step angle	θ_{step}	1.8°	±5%
Motor length	L_{motor}	65 mm	Dimension "L"
Mass	m	860 g	—

Source: Dings' Motion USA, NEMA 23 (size 23 / 57 mm) rotary stepper motor specifications [18].

Model Parameter Integration

Modeling the plant was achieved through use of the Dings' Motion 23H2065 datasheet, (Table 3) and the THK ballscrew catalog (model BIF1605-5RRG0+710LC5A) with a 16 mm shaft diameter and a 5 mm lead [37]. The load-side inertia is reflected to the motor shaft as a rotational inertia. The additional inertia for the milling surface, workholding, and stock material was not modeled in this plant. The inertia change between a physical mill and the representative Simulink model will result in more inertia on a future system. This is not expected to have substantial impact on the performance of the control system. A test was conducted by increasing the inertia of the plant in Simulink and comparing the results, shown in Figure 18.

Torque Constant The torque constant for the plant was found through the datasheet and experimentally by back-EMF method. To calculate the torque constant from the datasheet, the holding torque and the rated current was used in Equation (3). The provided current limit in the Dings' Motion 23H2065 Datasheet is an RMS value which is converted into a peak value in Equation (2).

Figure placeholder: `figures/ch2_inertia_sensitivity`

Alt text: Two stacked plots of the simulated response to the same position step with total inertia J, 2J, and 3J: position versus time on top and velocity versus time below.

Figure 18: Sensitivity of the simulated step response to the load inertia, with total inertia J, $2J$, and $3J$.

$$I_{\mathrm{pk}} = \sqrt{2}\, I_{\mathrm{rated}} = \sqrt{2}\,(3.0\,\mathrm{A}) = 4.243\,\mathrm{A}, \tag{2}$$

$$K_t = \frac{T_{\mathrm{h}}}{\sqrt{2}\, I_{\mathrm{rated}}} = \frac{1.60\,\mathrm{N\,m}}{\sqrt{2}\,(3.0\,\mathrm{A})} = 0.377\,\mathrm{N\,m\,A^{-1}}. \tag{3}$$

The factor of $\sqrt{2}$ in Equation (3) converts the rms rated current of the datasheet to its peak value, Equation (2) [18]. Under field-oriented control with $i_d = 0$, each phase current is a sinusoid of amplitude $|i_q|$, so the peak current I_{pk} is the quadrature current that produces the rated holding torque.

The back-EMF constant was measured by rotating the motor shaft at a constant speed with the windings open-circuited and recording the open-circuit phase voltage. With no current flowing, the phase voltage equals the back-EMF of Equation (18), whose amplitude is $\psi_m \omega_e = K_t\, \omega_m$ by Equations (19) and (27). The ratio of the peak phase voltage to the mechanical speed therefore gives the back-EMF constant K_E, which in SI units is numerically equal to K_t. The average of the measurements was $K_E = 0.4135\,\mathrm{V\,s\,rad^{-1}}$, 9.6 % above the datasheet value of Equation (3). The simulation uses the datasheet value. A true torque

constant 9.6 % above the design value raises the true velocity-loop input gain by the same fraction relative to b_0, which the observer absorbs into the total disturbance, as described in Section Discrete-Time First-Order ADRC.

Winding Resistance and Inductance The winding resistance and inductance are the per-phase datasheet values:

$$R = 0.9\,\Omega, \qquad L = 2.7\,\text{mH}. \tag{4}$$

Therefore the electrical time constant is

$$\tau_e = \frac{L}{R} = \frac{2.7\,\text{mH}}{0.9\,\Omega} = 3.0\,\text{ms}. \tag{5}$$

Simulation work uses the provided datasheet values of 2.7 mH in calculation. The inductance of the actual hybrid stepper motor was measured through a calculation of the impedance driven by a 1 kHz signal in series with a 15 Ω resistor. The values measured were within the expected tolerance range of 20% as listed in the datasheet. The values measured are:

Table 4: Measured versus datasheet stepper motor inductance (at 1 kHz)

Phase	Symbol	Measured	Datasheet nominal	Deviation from nominal
Phase A	L_A	3.205 mH	2.7 mH	+18.7 %
Phase B	L_B	3.209 mH	2.7 mH	+18.9 %

Source: Experimentally gathered at 1 kHz, compared against the Dings' Motion USA NEMA 23 (Size 23 / 57 mm) rotary stepper motor specifications [18].

The electrical time constant also limits how quickly the phase current can change, which bounds how quickly the torque can reverse. If only the voltage needed to sustain the peak current were applied, $R\,I_{\text{pk}} = 3.8\,\text{V}$, a change of current would settle in about

$3\tau_e = 9\,\mathrm{ms}$. The drive instead applies the full $48\,\mathrm{V}$ supply, so the current slews at up to $V_s/L \approx 17\,800\,\mathrm{A\,s^{-1}}$ when resistance and back-EMF are neglected. Reversing a phase current from $+I_{\mathrm{pk}}$ to $-I_{\mathrm{pk}}$ therefore takes at least $2I_{\mathrm{pk}}L/V_s \approx 0.48\,\mathrm{ms}$. At the maximum speed the back-EMF, $K_t\,\omega_m \approx 13.4\,\mathrm{V}$, reduces the available voltage and lengthens this time. Driving the winding with the full supply voltage is what allows the current loops to reach a bandwidth far above $1/\tau_e$.

Rotor Inertia The motor inertia J_m is the rotor inertia provided on the datasheet (Table 3):

$$J_m = 350\,\mathrm{g\,cm^2} = 3.5 \times 10^{-5}\,\mathrm{kg\,m^2}. \tag{6}$$

Viscous Damping The viscous damping coefficient B_m is not provided on the datasheet for the 23H2065, for the purpose of simulation the viscous damping coefficient was estimated at a value that is less than expected on the physical system. This results in a simulation that is inherently less damped than the physical plant will be. No measured or published value was available for this motor, so the value was chosen as an estimate comparable to values used for similar motors and kept low so that the simulated plant is harder to control than the physical one. With the total inertia of Equation (11), it gives a mechanical time constant of about $80\,\mathrm{ms}$. Because B_m enters the velocity loop only through the total disturbance of Equation (54), an error in this estimate is rejected by the observer and does not need to be identified.

$$B_m = 8 \times 10^{-4}\,\mathrm{N\,m\,s\,rad^{-1}}. \tag{7}$$

Ball Screw and Nut Inertia The screw shaft is modeled as a solid cylinder of outer diameter d and length L_s. With the catalog shaft mass per unit length $\lambda_s = 1.25\,\mathrm{kg\,m^{-1}}$ for the $d = 16\,\mathrm{mm}$ shaft, the shaft mass is $m_s = \lambda_s L_s$ [37] and

$$J_s = \frac{1}{2}\, m_s \left(\frac{d}{2}\right)^2 = \frac{1}{2}\, \lambda_s L_s \left(\frac{d}{2}\right)^2. \tag{8}$$

For $L_s = 0.71\,\mathrm{m}$,

$$J_s = \frac{1}{2}\,(1.25\,\mathrm{kg\,m^{-1}})(0.71\,\mathrm{m})(0.008\,\mathrm{m})^2 = 2.8400 \times 10^{-5}\,\mathrm{kg\,m^2}. \tag{9}$$

The nut translates as the screw turns. Its translating mass m_n is reflected to the screw shaft through the lead P_h, so that the kinetic energy of the nut equals that of an equivalent rotating inertia:

$$J_n = m_n \left(\frac{P_h}{2\pi}\right)^2 = (0.57\,\mathrm{kg}) \left(\frac{0.005\,\mathrm{m}}{2\pi}\right)^2 = 3.61 \times 10^{-7}\,\mathrm{kg\,m^2}. \tag{10}$$

Total Inertia The total inertia seen by the motor is the sum of the three components:

$$J = J_m + J_s + J_n = 3.5 \times 10^{-5} + 2.84 \times 10^{-5} + 3.61 \times 10^{-7} \approx 6.38 \times 10^{-5}\,\mathrm{kg\,m^2}. \tag{11}$$

Table 5: Plant parameters used in the simulation and source of parameter.

Parameter	Symbol	Source	Value
Torque constant	K_t	$T_\mathrm{h}/(\sqrt{2}\,I_\mathrm{rated})$	$0.377\,\mathrm{N\,m\,A^{-1}}$
Resistance	R	Dings' datasheet	$0.9\,\Omega$
Inductance	L	Dings' datasheet	$2.7\,\mathrm{mH}$
Motor inertia	J_m	Dings' datasheet	$3.5 \times 10^{-5}\,\mathrm{kg\,m^2}$
Viscous damping	B_m	Estimated	$8 \times 10^{-4}\,\mathrm{N\,m\,s\,rad^{-1}}$
Screw inertia	J_s	THK catalog, Eq. (8)	$2.84 \times 10^{-5}\,\mathrm{kg\,m^2}$
Nut inertia	J_n	THK catalog, Eq. (10)	$3.61 \times 10^{-7}\,\mathrm{kg\,m^2}$

Stepper Motor Model

To properly represent the control system and the expected functionality of the control system on the FPGA with the plant and PCB, a detailed model of the plant and PCB were created. The model is intended to reproduce the behavior of the physical system closely enough that controller performance in simulation is indicative of performance on physical hardware.

Electrical Model

The electrical model of the stepper motor converts the logic-level pulse-width modulation (PWM) drive signals produced by the controller into phase voltages, and the phase voltages into phase currents. These phase currents in turn produce the electromagnetic torque acting on the rotor. The Simulink model is split into two levels, the logic circuits that emulate an H-Bridge motor driver and the electrical dynamics of the stepper motor. Figure 19 shows the logic circuits diagram for the H-Bridge motor driver, and Figure 20 shows the subsystem that converts the motor drive voltage into phase currents.

Drive Stage Voltage Each phase of the stepper motor is driven independently with an H-Bridge to allow for control of the quadrature and direct vectors through a combination of each motor phase. For each phase $x \in \{a, b\}$, the controller supplies two Boolean signals, a

Figure placeholder: `figures/ch2_hbridge_logic`

Alt text: Screenshot of the Simulink H-Bridge logic for one phase: the Boolean PWM gate signal is converted to double, scaled by the supply voltage, and multiplied by plus or minus one as selected by the direction signal.

Figure 19: Simulink model of the H-Bridge drive stage for one phase.

Figure placeholder: `figures/ch2_stepper_electrical`

Alt text: Screenshot of the StepperElectrical subsystem: for each phase, the back-EMF and resistive voltage are subtracted from the applied voltage, scaled by one over L, and integrated by a forward Euler discrete-time integrator to give the phase current.

Figure 20: The `StepperElectrical` subsystem, which converts phase voltages into phase currents.

PWM gate signal $u_x[k] \in \{0, 1\}$, which indicates whether the bridge is conducting at sample k, and a direction signal $d_x[k] \in \{0, 1\}$, which selects the polarity of the applied voltage. Table 6 summarizes these signals. The voltage synthesis stage maps each pair of signals to a phase voltage through three operations:

- **Type conversion.** A Data Type Conversion block casts u_x from Boolean to double precision, so that all subsequent arithmetic is performed in floating point. Under this cast, a logical true maps to 1 and a logical false maps to 0. This is done as the stepper motor is not intended for HDL synthesis and is therefore modeled completely in double floating point.

- **Voltage scaling.** The control signal is multiplied by the motor source voltage, V_s, to model the H-Bridge connecting the phase to the motor voltage. This is achieved through a gain block which assumes an ideal H-Bridge with no propagation delay, ringing, or latency.

- **Polarity selection.** A Switch block with the criterion $d_x > 0$ passes the constant -1 when the direction signal is set and the constant $+1$ otherwise. This reflects the motor drive direction bit logic used on the physical H-Bridge drivers used to control the stepper motor.

Combining these operations, the voltage applied to phase x at sample k is

$$v_x[k] = V_s \, u_x[k] \, \sigma_x[k], \qquad \sigma_x[k] = \begin{cases} -1, & d_x[k] = 1, \\ +1, & d_x[k] = 0, \end{cases} \tag{12}$$

which can be represented compactly as $\sigma_x = 1 - 2d_x$. The phase voltage is able to take one of three levels, $v_x \in \{-V_s, 0, +V_s\}$.

When the gate signal u_x is low, Equation (12) applies zero volts across the winding regardless of the direction signal. This corresponds to the slow-decay mode of the physical H-Bridge, in which both low-side switches conduct and short the winding terminals together. With no applied voltage, the winding current changes under the winding resistance and back-EMF alone, as described by Equations (16) and (17) with $v_x = 0$, which produces a gradual braking torque on the rotor. The fast-decay mode, in which the winding current recirculates through the freewheeling diodes back into the supply, is not modeled. The simulated drive therefore always operates in slow decay during the off portion of each PWM period.

Dead time, switch on-resistance, and freewheeling diode drops are omitted from this model, as the model assumes these disturbances will be handled by the Active Disturbance Rejection Control system controlling the quadrature and direct currents during run time.

The simulation sample time $T_s = 5\,\mathrm{ns}$ is restricted to be much less than the PWM period T_{pwm} so that the model resolves individual PWM pulses. Over one PWM period, $20\,\mu\mathrm{s}$, with duty ratio D_x and constant direction, the average phase voltage is modeled as

$$\bar{v}_x = \frac{1}{T_{\mathrm{pwm}}} \int_{t}^{t+T_{\mathrm{pwm}}} v_x(\tau)\,\mathrm{d}\tau = V_s\, D_x\, \sigma_x. \tag{13}$$

Because the electrical time constant of the winding, shown in Equation (5), $\tau_e = 3\,\mathrm{ms}$, is 150 times greater than T_{pwm}, the winding behaves as a low-pass filter. The phase current consequently follows $\bar{v}_x$, with a superimposed ripple. Neglecting resistance and back electromotive force (EMF), the peak-to-peak ripple is approximately

$$\Delta i_x \approx \frac{V_s\, D_x(1 - D_x)\, T_{\mathrm{pwm}}}{L}, \tag{14}$$

which is largest at $D_x = 0.5$. This expression provides a check on whether the chosen PWM carrier frequency yields an acceptable current ripple for a given winding inductance

L. A PWM carrier frequency that is not sufficiently fast would introduce discrete current behavior into the stepper motor, leading to discrepancies in the measured versus expected current values in the quadrature and direct phase.

Substituting the chosen simulation values into Equation (14) the peak-to-peak ripple is calculated to be approximately:

$$\begin{aligned} \Delta i_x &\approx \frac{48 \times 0.5 \times (1 - 0.5) \times \left(\frac{1}{50000}\right)}{2.7 \times 10^{-3}} \\ \Delta i_x &\approx 89\,\mathrm{mA} \end{aligned} \tag{15}$$

A summary of the relevant electrical input signals is provided in Table 6.

Table 6: Input signals of the voltage synthesis stage.

Port	Signal	Type	Description
1	u_a (`Va`)	Boolean	PWM gate signal, phase A
2	u_b (`Vb`)	Boolean	PWM gate signal, phase B
3	d_a (`Va_direction`)	Boolean	Polarity select, phase A
4	d_b (`Vb_direction`)	Boolean	Polarity select, phase B

Phase Winding Model Each phase winding is modeled as a series resistance and inductance in series with a speed-dependent back-EMF, which is the standard representation of a two-phase permanent-magnet or hybrid stepper motor[7, 16, 22]. The `StepperElectrical` subsystem receives the phase voltages v_a and v_b, the electrical rotor speed ω_e, and the electrical rotor angle θ_e, and it implements

$$L\frac{\mathrm{d}i_a}{\mathrm{d}t} = v_a - R\,i_a + \psi_m\,\omega_e \sin\theta_e, \tag{16}$$

$$L\frac{\mathrm{d}i_b}{\mathrm{d}t} = v_b - R\,i_b - \psi_m\,\omega_e \cos\theta_e, \tag{17}$$

where i_a and i_b are the phase currents, R is the winding resistance, L is the winding self-inductance, and the back-EMF gain is the flux linkage ψ_m of Equation (27). The speed and angle inputs are the electrical quantities ω_e and θ_e of Equation (19). Defining the back-EMF voltages as

$$e_a = -\psi_m \, \omega_e \sin \theta_e, \qquad e_b = \psi_m \, \omega_e \cos \theta_e, \tag{18}$$

both equations take the circuit form $v_x = R\, i_x + L \, \mathrm{d}i_x/\mathrm{d}t + e_x$. The sine and cosine dependence places the two back-EMF waveforms in quadrature, consistent with the 90° electrical displacement between the phase windings.

Electromechanical Model

Although the simulation is implemented in the stationary phase frame of Equations (16) and (17), the controller regulates the direct and quadrature currents, so the stepper motor is also described as a two-phase permanent-magnet machine in the rotor-fixed d–q frame [7, 16]. The motor has $N_r = 50$ rotor teeth, therefore one mechanical revolution corresponds to N_r electrical periods, and the electrical angle and speed are

$$\theta_e = N_r \, \theta_m, \qquad \omega_e = N_r \, \omega_m, \tag{19}$$

where θ_m and ω_m are the mechanical angle and angular velocity of the rotor. The phase currents are mapped into the rotor frame by the Park transform, which for a two-phase machine is a rotation about the electrical angle [7, 25],

$$i_d = i_a \cos \theta_e + i_b \sin \theta_e, \qquad i_q = -i_a \sin \theta_e + i_b \cos \theta_e, \tag{20}$$

and the phase voltages and back-EMF voltages transform in the same way. With equal direct and quadrature inductances ($L_d = L_q = L$, a non-salient rotor), applying Equation (20) to Equations (16) and (17) gives the stator voltage equations

$$v_d = R\, i_d + L\, \frac{di_d}{dt} - N_r\, \omega_m\, L\, i_q, \tag{21}$$

$$v_q = R\, i_q + L\, \frac{di_q}{dt} + N_r\, \omega_m\, L\, i_d + N_r\, \omega_m\, \psi_m, \tag{22}$$

and the rotor dynamics are

$$J\, \frac{d\omega_m}{dt} = T_e - B_m\, \omega_m - T_L, \tag{23}$$

where T_e is the electromagnetic torque, B_m is the viscous damping coefficient, T_L is the load torque, and $J = J_m + J_s + J_n$ is the total inertia from Equation (11).

Phase Current Integration Within the `StepperElectrical` subsystem, Equations (16) and (17) are integrated with Discrete-Time Integrator blocks configured with the forward Euler method, the same method used for the rotor speed and angle in the mechanical model. For each phase, the subsystem forms the voltage across the winding inductance, $v_x - R\, i_x - e_x$, scales it by $1/L$ to obtain the rate of change of current, and integrates the result to obtain the phase current. Combining these operations, the phase current at sample $k+1$ is

$$i_x[k+1] = i_x[k] + \frac{T_s}{L}\Big(v_x[k] - R\, i_x[k] - e_x[k]\Big), \qquad x \in \{a, b\}, \tag{24}$$

where $v_x[k]$ is the synthesized phase voltage of Equation (12) and $e_x[k]$ is the back-EMF voltage of Equation (18), evaluated with the electrical rotor speed and angle at sample k. Grouping i_x terms gives

$$i_x[k+1] = \left(1 - \frac{T_s}{\tau_e}\right) i_x[k] + \frac{T_s}{L}\Big(v_x[k] - e_x[k]\Big), \qquad \tau_e = \frac{L}{R}, \tag{25}$$

where τ_e is the electrical time constant of the winding from Equation (5). The single pole at $z = 1 - T_s/\tau_e$ is the forward Euler image of the continuous-time winding pole at $s = -1/\tau_e$, so the discretization is stable for $T_s < 2\tau_e$. The electrical time constant is typically far shorter than the mechanical time constant τ_m, so this condition, rather than the mechanical stability condition, sets the upper bound on the sample time. In practice the sample time is set by the stricter requirement that $T_s \ll T_{\text{pwm}}$ in order to resolve individual PWM pulses. Since $T_{\text{pwm}} \ll \tau_e$, the ordering $T_s \ll T_{\text{pwm}} \ll \tau_e$ holds, and the current integration is both stable and accurate with a wide margin.

The forward Euler method is also well suited to the PWM drive. During a single PWM pulse the applied voltage is constant, and when resistance and back-EMF are neglected, as in the derivation of Equation (14), the current changes at a constant rate. Forward Euler integrates a constant rate exactly, therefore the simulated phase current reproduces the triangular ripple of Equation (14), with the timing of each pulse edge resolved to the nearest sample.

All forward Euler integrators have no direct feedthrough, and $i_x[k+1]$ depends only on signals at sample k. The back-EMF term uses the rotor speed and angle at sample k, which are integrator outputs from the mechanical model, and the electromagnetic torque at sample k uses the phase currents at sample k. No signal in the electrical–mechanical loop depends on another signal at the same sample, and no algebraic loop is formed.

Flux Linkage and Torque Equation The flux linkage ψ_m is the peak flux linkage between the rotor magnets and one stator phase. ψ_m sets the back-EMF, the term $N_r\,\omega_m\,\psi_m$ in Eq. (22), which opposes the applied voltage and grows linearly with speed. The units of ψ_m

are V s.

The electromechanical power is the product of the back-EMF and the torque-producing current. Because the two-phase Park transform of Equation (20) is solely a rotation, it is both amplitude- and power-invariant, and the power into the back-EMF is $P_e = (N_r\,\omega_m\,\psi_m)\,i_q$, and since $P_e = T_e\,\omega_m$, the torque is

$$T_e = N_r\,\psi_m\,i_q = K_t\,i_q, \tag{26}$$

which defines the motor torque constant

$$K_t = N_r\,\psi_m \quad \Longleftrightarrow \quad \psi_m = \frac{K_t}{N_r}. \tag{27}$$

The motor model assumes a non-salient rotor, $L_d = L_q = L$, which is the standard assumption for hybrid stepper motors [16, 22]. The near-equal phase inductances of Table 4 are consistent with this assumption, since $L_d = L_q$ is dependent on the variation of inductance with rotor position rather than the match between phases. With this simplified model of the motor, reluctance torque and detent torque are omitted and the direct-axis current i_d produces no torque and is regulated to zero [22].

For the 23H2065 motor, with $K_t = 0.377\,\mathrm{N\,m\,A^{-1}}$ and $N_r = 50$,

$$\psi_m = \frac{0.377\,\mathrm{N\,m\,A^{-1}}}{50} = 7.54 \times 10^{-3}\,\mathrm{V\,s}. \tag{28}$$

Table 7: Parameters of the electrical model.

Symbol	Simulink Block	Description
V_s	Voltage scaling gain	Motor supply voltage, 48 V
T_{pwm}	Phase A and B PWM Generators	PWM period, 20 µs (50 kHz)
R	`StepperElectrical`	Winding resistance, 0.9 Ω (Ω)
L	`StepperElectrical`	Winding inductance, 2.7 mH
ψ_m	`StepperElectrical`	Back-EMF gain, 7.54×10^{-3} V s

Mechanical Model

The mechanical model of the stepper motor converts the phase currents produced by the electrical model into electromagnetic torque and integrates the resulting torque balance to obtain the rotor speed and angle. These mechanical states are returned to the electrical model, where these states affect the magnitude and phase of the back-EMF, and are exported from the motor model to compute encoder pulses. The Simulink implementation is divided into three subsystems, shown in Figure 21: the `Te` subsystem, which computes the electromagnetic torque from the phase currents and rotor angle; the `StepperMechanical` subsystem, which implements the rotor dynamics of Equation (23); and the `rotor angle` subsystem, which integrates the rotor speed to obtain the mechanical and electrical rotor angles. As with the electrical model, all signals are represented in double-precision floating point, as the motor model is not intended for HDL synthesis.

Electromagnetic Torque The `Te` subsystem receives the phase currents i_a and i_b from the electrical subsystem, `StepperElectrical`, and the mechanical rotor angle θ_m from the `rotor angle` subsystem. Because the electrical model is calculated in the stationary phase frame instead of the rotor-fixed d–q frame, the torque is computed directly from the phase A and phase B currents as

$$T_e = K_t \left[-i_a \sin(\theta_e) + i_b \cos(\theta_e) \right], \tag{29}$$

Figure placeholder: `figures/ch2_mechanical_model`

Alt text: Screenshot of the stepper motor mechanical model showing three subsystems: Te computes electromagnetic torque from the phase currents and rotor angle, StepperMechanical integrates the torque balance to rotor speed, and rotor angle integrates speed to the mechanical and electrical angles.

Figure 21: Mechanical model of the stepper motor: the `Te`, `StepperMechanical`, and `rotor angle` subsystems.

where the electrical angle $\theta_e = N_r\theta_m$ is formed within the subsystem from the supplied mechanical angle.

Equation (29) follows from the power balance of the back-EMF sources. Using the back-EMF voltages of Equation (18), the electrical power converted by the two phase windings is

$$P_e = e_a i_a + e_b i_b = \psi_m \omega_e \left(-i_a \sin\theta_e + i_b \cos\theta_e\right). \tag{30}$$

Substituting the electrical speed $\omega_e = N_r\omega_m$ from Equation (19) and equating P_e to the mechanical power $T_e\omega_m$ gives

$$T_e = N_r \psi_m \left(-i_a \sin\theta_e + i_b \cos\theta_e\right). \tag{31}$$

With the back-EMF gain equal to the permanent-magnet flux linkage ψ_m, the leading factor becomes $N_r\psi_m = K_t$ by Equation (27), which recovers Equation (29).

The bracketed term in Equation (29) is the quadrature current i_q of the Park transform

in Equation (20), so the phase-frame expression reduces to $T_e = K_t i_q$, consistent with Equation (26). The two formulations are therefore equivalent. The phase-frame form is used in simulation so that the torque responds to the instantaneous phase currents, including the PWM current ripple estimated in Equation (15), rather than to idealized d–q currents. As a result, the simulated torque carries a ripple on the order of $K_t \Delta i_x \approx 0.034\,\mathrm{N\,m}$ at the PWM carrier frequency, which the controller must tolerate. Detent torque is neglected, consistent with the electromechanical model.

By driving the stepper motor using the phase currents, additional modeling was able to be done on the stepper motor. To test the stepper motor functions separately from the control system designed in the present work, a separate test model was created that drove the stepper motor like a traditional stepper motor performing full stepping to confirm proper functionality of the stepper motor. The result is shown in Figure 22.

Figure placeholder: `figures/ch2_fullstep_validation`

Alt text: Plots from the full-step test model: the alternating phase A and phase B drive currents and the resulting rotor angle advancing in discrete 1.8 degree steps.

Figure 22: Validation of the stepper motor model driven with conventional full-step commands.

Rotor Dynamics The `StepperMechanical` subsystem, shown in Figure 23, implements the rotor torque balance of Equation (23). The subsystem receives the electromagnetic torque T_e and the load torque T_L and produces the mechanical rotor speed ω_m through four operations.

- **Torque summation.** A Sum block with signs $(+, -, -)$ forms the net accelerating torque $T_e - B_m\omega_m - T_L$. The electromagnetic torque drives the rotor, while the viscous damping torque and the load torque oppose it.
- **Inertial scaling.** A gain block of value $1/J$ converts the net torque into angular acceleration, where J is the total inertia from Equation (11).
- **Integration.** A Discrete-Time Integrator configured with the forward Euler method, with transfer function $KT_s/(z-1)$ and unity gain K, integrates the angular acceleration to obtain the rotor speed.
- **Damping feedback.** The rotor speed is multiplied by the viscous damping coefficient B_m and returned to the negative input of the Sum block, closing the damping loop.

Figure placeholder: `figures/ch2_stepper_mechanical`

Alt text: Screenshot of the StepperMechanical subsystem: a sum block forms electromagnetic torque minus damping torque minus load torque, a one-over-J gain gives angular acceleration, and a discrete-time integrator gives rotor speed, which is fed back through the damping gain Bm.

Figure 23: The `StepperMechanical` subsystem, which implements the rotor torque balance of Equation (23).

Combining these operations, the rotor speed at sample $k+1$ is

$$\omega_m[k+1] = \omega_m[k] + \frac{T_s}{J}\Big(T_e[k] - B_m\omega_m[k] - T_L[k]\Big), \tag{32}$$

which is the forward Euler discretization of Equation (23). Collecting terms in ω_m yields the discrete transfer function from net driving torque to rotor speed,

$$\frac{\omega_m(z)}{T_e(z) - T_L(z)} = \frac{T_s/J}{z - (1 - T_s/\tau_m)}, \qquad \tau_m = \frac{J}{B_m}, \tag{33}$$

where τ_m is the mechanical time constant of the rotor, $\tau_m = J/B_m \approx 80\,\text{ms}$. With $T_s = 5\,\text{ns}$, the ratios $T_s/\tau_m \approx 6 \times 10^{-8}$ and $T_s/\tau_e \approx 2 \times 10^{-6}$ place both discretizations many orders of magnitude inside their stability limits. This is the forward Euler discretization of the continuous-time plant, $1/(Js + B_m)$, with its pole at $s = -1/\tau_m$ is mapped to $z = 1 - T_s/\tau_m$. The discretization is stable for $|1 - T_s/\tau_m| < 1$, which requires $T_s < 2\tau_m$. Sample time is chosen much shorter than the PWM period in order to resolve individual pulses, and the mechanical time constant is much longer than the PWM period, therefore this condition is satisfied and the discretization error of the forward Euler method is negligible.

The forward Euler method was selected because it has no direct feedthrough, the integrator output at sample k depends only on inputs up to sample $k - 1$. This prevents the damping feedback path through B_m from forming an algebraic loop with the Sum block. The same one-sample delay decouples the electrical and mechanical models, since a change in T_e at sample k affects ω_m only at sample $k + 1$. Given that $T_s \ll \tau_m$, this delay has no meaningful effect on the simulated dynamics.

The load torque T_L is supplied externally of the motor model, which allows step or time-varying load disturbances to be applied during simulation such as a spring modeled as an increasing linear load. Given that T_L enters the torque balance directly and is not measured by the controller, it appears to the Active Disturbance Rejection Control system as part of the total disturbance to be estimated and rejected. The measurement of the applied T_L through the use of the load cells is intended only for verification of system functionality. The rotor speed ω_m is routed out of the motor model, to the `rotor angle` subsystem, and through a gain of N_r to form the electrical speed ω_e, which is supplied to the `StepperElectrical`

subsystem.

Rotor Angle The `rotor angle` subsystem, shown in Figure 24, integrates the rotor speed to obtain the rotor position. A second forward Euler Discrete-Time Integrator produces the mechanical angle, and a gain of N_r, labeled `RT` in the model, converts it to the electrical angle:

$$\theta_m[k+1] = \theta_m[k] + T_s\,\omega_m[k], \qquad \theta_e[k] = N_r\,\theta_m[k]. \tag{34}$$

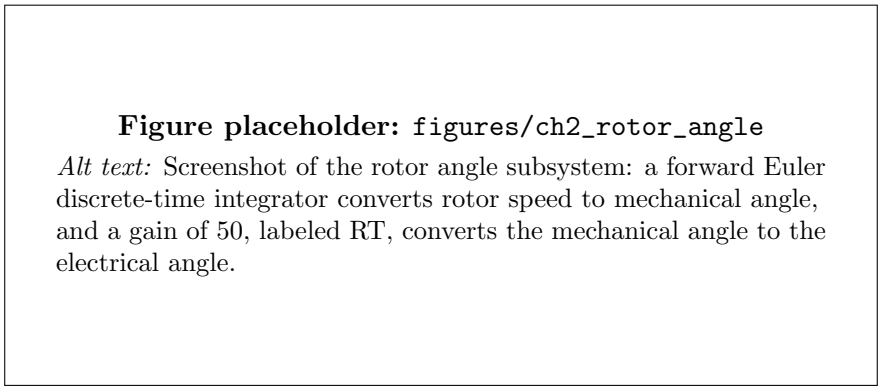


Figure 24: The `rotor angle` subsystem.

The mechanical angle θ_m is supplied to the `Te` subsystem for the torque calculation of Equation (29) and is exported through output port 5 of the motor model for the encoder emulation. The electrical angle θ_e is supplied to the `StepperElectrical` subsystem, where it sets the phase of the back-EMF terms in Equations (16) and (17). The angle is not wrapped to a single revolution. Accumulating θ_m without bound preserves the multi-turn rotor position required by the encoder emulation, and wrapping is unnecessary for the torque and back-EMF calculations because the sine and cosine functions are periodic.

Together, the `StepperMechanical` and `rotor angle` subsystems form a cascade of two discrete integrators from net torque to rotor position. Combined, the mechanical and electrical models describe the complete stepper motor by the state vector $[\,i_a,\ i_b,\ \omega_m,\ \theta_m\,]^{\mathsf{T}}$.

The state vector advances once per sample by Equations (24), (32), and (34).

The signals of the mechanical model are summarized in Table 8, and its parameters in Table 9.

Table 8: Signals of the mechanical model.

Subsystem	Signal	Direction	Description
`Te`	i_a, i_b	Input	Phase currents (A)
	θ_m	Input	Mechanical rotor angle (rad)
	T_e	Output	Electromagnetic torque (N m)
`StepperMechanical`	T_e	Input	Electromagnetic torque (N m)
	T_L	Input	Load torque, motor input port 3 (N m)
	ω_m	Output	Rotor speed, motor output port 4 (rad s^{-1})
`rotor angle`	ω_m	Input	Rotor speed (rad s^{-1})
	θ_m	Output	Mechanical angle, motor output port 5 (rad)
	θ_e	Output	Electrical angle, motor output port 3 (rad)

Table 9: Parameters of the mechanical model.

Symbol	Simulink Block	Description
K_t	`Te`	Torque constant, 0.377 N m A^{-1}
N_r	`RT` gain	Number of rotor teeth, 50
J	`1/J` gain	Total rotor and load inertia (kg m^2)
B_m	`Bm` gain	Viscous damping coefficient (N m s rad^{-1})
T_s	Discrete-Time Integrators	Simulation sample time (s)

PCB Model

The PCB model emulates the measurement interface between the stepper motor and the FPGA. The stepper motor model produces the rotor angle and phase currents as double-precision signals, while the physical controller receives these quantities only through the sensors and converters on the PCB: a quadrature encoder for rotor position, shunt

amplifiers sampled by ADCs for the phase currents, and an ADC for the load cell. The PCB model converts the motor outputs into the same digital signals, with the same data types, resolution, and offsets, that the FPGA receives from the board. This allows the decoding logic, fixed-point data types, and measurement quantization of the controller to be exercised in simulation with the same form of inputs it will process in hardware, in the form the HDL-compatible decoding logic expects.

Encoder Emulation

Rotor position is measured on the physical system with a 5000-line incremental quadrature encoder mounted to the motor shaft. The encoder produces two square-wave channels, A and B, offset by a quarter cycle, and an index channel, Z, that pulses once per revolution [17]. Processing every edge of both channels yields four counts per line, so one mechanical revolution corresponds to

$$N_c = 4N_{\text{line}} = 4 \times 5000 = 20000 \tag{35}$$

counts. The `Convert Mechanical Angle to Encoder Pulses` subsystem, shown in Figure 25 , receives the mechanical rotor angle θ_m from the `rotor angle` subsystem of the stepper motor model and generates the three Boolean encoder channels through four operations.

- **Angle scaling.** A gain block converts the mechanical angle from radians to encoder counts by multiplying by $N_c/2\pi$.
- **Integer conversion.** A Data Type Conversion block casts the scaled angle from double precision to a signed 32-bit integer, quantizing the continuous mechanical

angle to a whole number of encoder counts. This count models an ideal encoder accumulation. All three output channels are derived from this count.

- **Quadrature generation.** Bitwise AND blocks with the masks `0x1` and `0x2` extract the two least significant bits of the count. Channel B is taken directly from the second bit, and channel A is formed as the exclusive OR of the first and second bits.
- **Index generation.** A Math Function block computes the count modulo N_c, and a comparison block asserts the index channel when the result is zero.

Combining these operations, the encoder count at sample k is

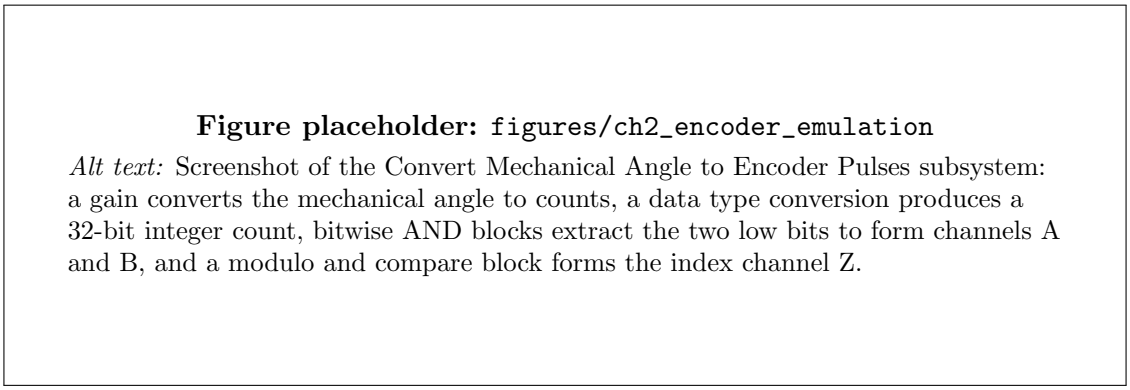


Figure 25: The `Convert Mechanical Angle to Encoder Pulses` subsystem.

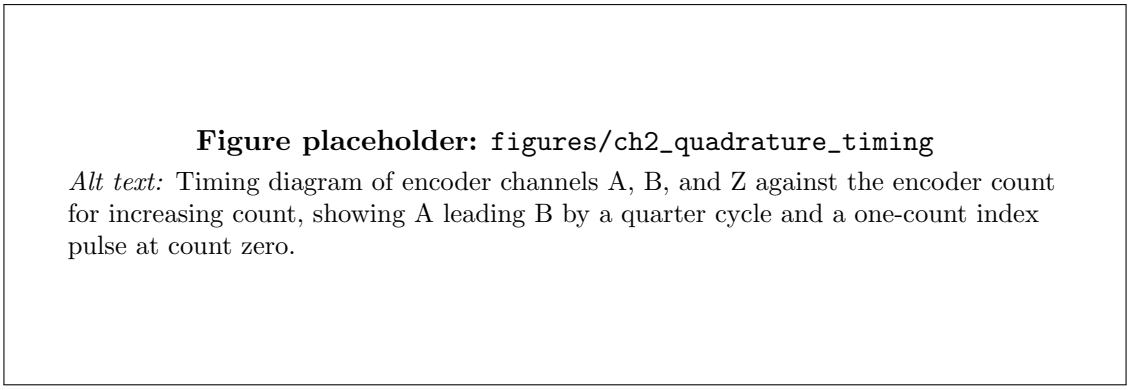


Figure 26: Quadrature encoder waveforms generated from the encoder count.

Figure placeholder: `figures/ch2_encoder_logic`

Alt text: Logic diagram of the encoder emulation: bit 0 and bit 1 of the encoder count feed an exclusive-OR gate that produces channel A, bit 1 drives channel B directly, and a modulo-20000 comparison produces the index channel Z.

Figure 27: Logic that generates the quadrature channels from the two low bits of the encoder count.

$$n[k] = \mathcal{Q}\left(\frac{N_c}{2\pi}\,\theta_m[k]\right), \tag{36}$$

where $\mathcal{Q}$ denotes the integer conversion of the Data Type Conversion block, rounding towards $-\infty$. Writing $n_0[k]$ and $n_1[k]$ for the first and second bits of $n[k]$, the encoder channels are

$$B[k] = n_1[k], \qquad A[k] = n_0[k] \oplus n_1[k], \qquad Z[k] = \begin{cases} 1, & n[k] \bmod N_c = 0, \\ 0, & \text{otherwise.} \end{cases} \tag{37}$$

Equation (37) converts the two low bits of the binary count into the two-bit Gray code produced by a quadrature encoder, summarized in Table 10 and Figure 26, with the generating logic in Figure 27. As the count increases, the channel state advances through the sequence $(A, B) = (0, 0) \to (1, 0) \to (1, 1) \to (0, 1)$, such that channel A leads channel B by a quarter cycle. When the count decreases, the sequence reverses and channel B leads. Exclusively one channel changes per count, which is the property the decoder relies

on to determine the direction of rotation. The count is stored in a two's complement form where the two low bits continue the same four-state cycle as the count passes through zero and becomes negative. A reversal of direction across the zero position therefore produces a continuous quadrature sequence with no discontinuities.

Table 10: Quadrature states generated from the encoder count.

$n \bmod 4$	n_1	n_0	A	B
0	0	0	0	0
1	0	1	1	0
2	1	0	1	1
3	1	1	0	1

The index channel is asserted for a single count whenever the rotor is at the reference zero position or a whole number of revolutions from it, which matches the once-per-revolution index pulse of the physical encoder. With the mechanical angle not being wrapped to a single revolution, the count accumulates the multi-turn position of the rotor. The signed 32-bit count can represent approximately $2^{31}/N_c \approx 107{,}000$ revolutions before overflow, which is far beyond the travel of any simulation, or practical physical system. The physical system is restricted to at most ≈ 142 revolutions.

The resolution of the encoder is one count, or

$$\Delta\theta_m = \frac{2\pi}{N_c} = 3.14 \times 10^{-4}\,\text{rad} \quad (0.018^\circ). \tag{38}$$

The emulated encoder also places an upper limit on the rotor speed that can be represented without loss of counts. From Equation (34), the mechanical angle advances by $T_s\,\omega_m$ each sample, so the count advances by approximately $N_c T_s \omega_m / 2\pi$ counts per sample. If the count advances by two or more in a single sample, both channels change together and the direction of motion cannot be determined from the channel states. The emulated

channels therefore remain valid only while

$$|\omega_m| < \frac{2\pi}{N_c T_s} = \frac{2\pi}{20000 \times 5\,\text{ns}} \approx 6.2832 \times 10^4\,\text{rad}\,\text{s}^{-1} \quad (600\,000\,\text{rpm}). \tag{39}$$

This limit is an artifact of updating the rotor angle in discrete steps. The physical encoder produces continuous edges, and the FPGA samples them at its system clock, which is many times faster than the maximum count rate of the motor. At the maximum speed of $35.5\,\text{rad}\,\text{s}^{-1}$, counts arrive every $8.85\,\mu\text{s}$, or about 1770 simulation steps, so the condition of Equation (39) is met with a margin of more than three orders of magnitude.

Table 11: Signals in the encoder subsystem.

Signal	Type	Direction	Description
θ_m	double	Input	Mechanical rotor angle (rad)
n	int32	Internal	Accumulated encoder count
A (`enc_a`)	Boolean	Output	Quadrature channel A
B (`enc_b`)	Boolean	Output	Quadrature channel B
Z (`enc_z`)	Boolean	Output	Index, once per revolution

Table 12: Parameters of the encoder emulation.

Symbol	MATLAB Variable	Description
N_{line}	—	Encoder lines per revolution, 5000
N_c	`CountsPerRev`	Counts per mechanical revolution, 20000
N_c/N_r	`electricalCountsPerRev`	Counts per electrical revolution, 400
$\Delta\theta_m$	`mechanicalRadiansPerCount`	Mechanical angle per count, $3.14 \times 10^{-4}\,\text{rad}$

Current Sensing

The phase currents are measured on the physical system with a shunt resistor in series with each winding. The voltage across the shunt resistor is amplified by an INA241 current-

sense amplifier and digitized by a 16-bit MAX11162 ADC. ADC[2, 36]. To account for the winding current direction reversals, the amplifier output is offset to half of the ADC reference voltage, so that both current polarities fall within the input range of the ADC. One `ADC Emulator` subsystem, shown in Figure 28 , is used for each phase. It receives the phase current i_x from the stepper motor model and produces the 16-bit ADC reading through four operations.

- **Shunt and amplifier gain.** A gain block multiplies the phase current by the product of the shunt resistance and the amplifier gain, $K_{cs} = R_{sh}\, G_{cs}$, converting current into the amplifier output voltage. The shunt resistor and amplifier are modeled as an ideal gain with no bandwidth limit, gain error, or offset error.

- **Amplifier offset.** A Sum block adds the constant offset $V_{\text{off}} = V_{\text{ref}}/2$, which models the reference voltage applied to the amplifier output as specified in the INA241 datasheet [36]. On the PCB V_{off} is referenced off of the V_{ref}, keeping the INA241 and ADC voltage linked.

- **Voltage to ADC scaling.** A gain block converts the amplifier output voltage into ADC counts by multiplying by $K_{\text{adc}} = 2^N/V_{\text{ref}}$, where $N = 16$, the ADC resolution in bits.

- **Quantization.** A Data Type Conversion block casts the result from double precision to the unsigned 16-bit type `fixdt(0,16,0)`, producing the integer reading the ADC reports to the FPGA. The ADC quantization simulation clips at the same voltage level as the physical ADC. The conversion rounds to the nearest code, matching the MAX11162, such that the mid-scale (2.5 V) corresponds to `0x8000` and bit transitions occur at ± 0.5 counts [2]. The quantization error is therefore at most half of one count.

Figure placeholder: `figures/ch2_adc_emulator`

Alt text: Screenshot of the ADC Emulator subsystem for one phase: the phase current is multiplied by the shunt and amplifier gain, offset by 2.5 volts, scaled to ADC counts, and rounded and saturated to an unsigned 16-bit reading.

Figure 28: The `ADC Emulator` subsystem for one phase.

Combining these operations, the amplifier output voltage and ADC reading for phase x at sample k are

$$v_{cs,x}[k] = K_{cs}\, i_x[k] + V_{\text{off}}, \qquad N_x[k] = \mathcal{Q}_{16}(K_{\text{adc}}\, v_{cs,x}[k])\,, \tag{40}$$

where $\mathcal{Q}_{16}$ denotes the conversion to an unsigned 16-bit integer. Substituting $V_{\text{off}} = V_{\text{ref}}/2$ gives

$$N_x[k] = \mathcal{Q}_{16}\big(K_{\text{adc}} K_{cs}\, i_x[k] + 2^{N-1}\big)\,, \tag{41}$$

so that zero current maps to the mid-point reading $2^{N-1} = 32768$, positive current maps above mid-point, and negative current maps below it. The input range of the ADC, $0 \le v_{cs,x} < V_{\text{ref}}$, sets the measurable current range and the current resolution as

$$|i_x| < \frac{V_{\text{ref}}}{2K_{cs}} = 5.0\,\text{A}, \qquad \Delta i = \frac{V_{\text{ref}}}{2^N K_{cs}} = 153\,\mu\text{A}. \tag{42}$$

Currents outside this range drive the amplifier and INA241 output beyond the input

range of the ADC, and the reading clips at 0 or $2^N - 1$. The resolution is finer than the PWM current ripple of approximately 89 mA from Equation (15), which spans roughly 585 ADC counts. The ADC therefore resolves the ripple itself, and the measured current depends on the point in the PWM period at which the sample is taken. To minimize the effect of the PWM ripple current, the ADC samples on the physical system should be aligned with the center point of the PWM output [9]. At the center of the on-time the instantaneous current equals its average over the PWM period, so a center-aligned sample carries no ripple error. The present model does not synchronize the ADC samples with the PWM period, so the decoded currents carry an error of up to half of the 89 mA peak-to-peak ripple, depending on where in the PWM period the sample falls.

The as-built PCB uses a 0.1 Ω shunt with a $10\,\mathrm{V\,V^{-1}}$ INA241 variant, which limits the measurable current to ±2.5 A per phase, below the 4.243 A peak current of the 23H2065 motor from Equation (2). Above this limit the measured current clips, and the current loop would continue to raise the applied voltage while the actual current exceeds its reference. The simulation therefore models a revised current-sensing stage that uses the INA241A4 ($100\,\mathrm{V\,V^{-1}}$) with a 5 mΩ shunt, giving $K_{cs} = 0.5\,\mathrm{V\,A^{-1}}$ [36]. The INA241 gain is specified as linear for outputs from 50 mV above ground to 200 mV below its supply, so with the 2.5 V offset the linear range is ±4.6 A, which covers the 4.243 A peak current and half of the 89 mA ripple. The revised stage also reduces the shunt dissipation at $3\,\mathrm{A_{rms}}$ from 0.9 W to 45 mW per phase. The revision changes two components on the PCB and is discussed further in Chapter 4.

The controller recovers the phase current by inverting Equation (41),

$$\hat{\imath}_x[k] = \frac{N_x[k] - 2^{N-1}}{K_{\mathrm{adc}} K_{cs}}. \tag{43}$$

The amplifier and ADC are modeled as ideal. Amplifier bandwidth, common-mode transients at the PWM switching edges, the input filters, and ADC noise, nonlinearity, and conversion latency are omitted. Unlike the motor-side disturbances, errors in the current measurement are not rejected by the Active Disturbance Rejection Control system. The current loop regulates the measured current, so a gain or offset error in the measurement becomes an error in the actual phase current. For this reason, the shunt resistance and amplifier offset used in the final system should be calibrated values measured on the PCB rather than nominal values. For example, the MAX11162 specifies an offset error of up to ± 3.5 counts, or $\pm 0.53\,\mathrm{mA}$ with the revised gain, which is larger than its quantization error [2].

Table 13: Signals of the current sensing emulation.

Signal	Type	Direction	Description
i_a, i_b	double	Input	Phase currents from the stepper motor model (A)
$v_{cs,x}$	double	Internal	Amplifier output voltage (V)
N_a, N_b	uint16	Output	ADC readings, PCB model output ports 1 and 2

Table 14: Parameters of the current sensing emulation.

Symbol	MATLAB Variable	Description
R_{sh}	`Shunt_Resistance`	Shunt resistance, $5\,\mathrm{m\Omega}$
G_{cs}	`Shunt_Amp_Gain`	Amplifier gain, $100\,\mathrm{V\,V^{-1}}$ (INA241A4)
K_{cs}	`ADC_Emulated_conversion_factor`	Current-to-voltage gain, $0.5\,\mathrm{V\,A^{-1}}$
V_{off}	`Shunt_Amp_Offset`	Amplifier output offset, 2.5 V
V_{ref}	`ADC_VREF`	ADC reference voltage, 5 V
N	`ADC_Precision`	ADC resolution, 16 bits
K_{adc}	`VoltagetoADCValue`	Voltage-to-count gain, 13 107.2 counts/V

Load Cell Emulation

[**Note:** I'm working on writing this section. This will be done ASAP now that the bugs in it have been resolved.]

Control System

The control system regulates the position of the stepper motor through field-oriented control, in which the phase currents are measured, transformed into the rotor-fixed d–q frame, and regulated so that the motor produces torque only through the quadrature current [6, 7]. The controller is implemented in Simulink in fixed-point arithmetic to allow HDL code for the FPGA to be synthesized. Every input provided is in the form produced by the PCB model in Section PCB Model: unsigned 16-bit ADC readings for the phase currents and Boolean quadrature channels from the encoder. Every output is a Boolean PWM and direction signal for each H-Bridge, as described in Section 2.

Control Architecture

The motor is controlled by a cascade of three loops, shown in Figure 29 . The position loop is a proportional controller with gain $K_{P\theta}$, which produces a velocity reference. The velocity loop, a first-order Active Disturbance Rejection Controller (ADRC), computes the quadrature current command i_q^*. Two current loops, also first-order ADRCs, compute the direct and quadrature voltage commands v_d^* and v_q^*. The signal path through the controller consists of five stages [1].

- **Feedback processing.** The ADC readings are decoded into phase currents in amps, and the encoder channels are decoded into the mechanical position which is then converted to the electrical angle, and the rotor velocity.
- **Position loop.** The position error is multiplied by $K_{P\theta}$ to form the velocity reference ω_m^*.

- **Velocity loop.** The velocity ADRC computes the quadrature current command i_q^* from ω_m^* and the measured velocity.
- **Current loops.** The measured phase currents are transformed into i_d and i_q with the Park transform, and two current ADRCs compute v_d^* and v_q^* that drive i_d to zero and i_q to i_q^*.
- **Voltage output.** The inverse Park transform converts v_d^* and v_q^* into the phase voltage commands, which the PWM generators convert into the gate and direction signals of each H-Bridge, modeling the FPGA outputs.

Figure placeholder: `figures/ch2_foc_controller_top`

Alt text: Screenshot of the Stepper Motor FOC Controller subsystem showing the ADC decoders, encoder decoder, position controller, velocity ADRC, Park transform, two current ADRCs, inverse Park transform, and PWM generators connected in a cascade.

Figure 29: Top level of the `Stepper Motor FOC Controller` subsystem.

The controller runs at three rates. The feedback processing (encoder decoding, velocity estimation, ADC decoding, and Park transforms), the position loop, and the PWM generators run at the 5 ns simulation step, which represents the 200 MHz FPGA logic clock. The current loops execute every $T_i = 20\,\mu s$, which equals the PWM period, so that each current loop is updated once per PWM cycle. The velocity loop executes every $T_\omega = 400\,\mu s$, based on the limitations in Section Velocity Loop. The position loop is a pure gain, and its output is sampled by the velocity loop, so it acts at the velocity-loop rate. The closed-loop and observer bandwidths are summarized in Table 15. Each control loop is slower than the loop it commands, the current loop is 25 times faster than the velocity loop, and the velocity

loop is 4.4 times faster than the position loop. This separation allows each inner current loop to be treated as an ideal actuator by the loop around it, with the residual inner-loop dynamics absorbed by the observer of the outer ADRC.

Table 15: Closed-loop bandwidth ω_{CL} and observer bandwidth ω_{ESO} of the control loops.

Loop	Controller	ω_{CL}	ω_{ESO}
Position	Proportional	$90\,\mathrm{rad\,s^{-1}}$	—
Velocity	First-order ADRC	$400\,\mathrm{rad\,s^{-1}}$	$1600\,\mathrm{rad\,s^{-1}}$
Current (d and q)	First-order ADRC	$10\,000\,\mathrm{rad\,s^{-1}}$	$20\,000\,\mathrm{rad\,s^{-1}}$

The controller gains and ADRC coefficients are supplied to the controller subsystem as inputs rather than as internal constants, this is done so that on the FPGA implementation these inputs can be held in registers and configured at run time. The controller can be re-tuned without regenerating or re-synthesizing the HDL. This allows for the control system to be retuned for different motors without restructuring the entire system.

Feedback Signal Processing

Current Decoding The `adc_phase_a_decoder` and `adc_phase_b_decoder` subsystems invert the current sensing chain of Section Current Sensing. Each decoder receives the unsigned 16-bit ADC reading N_x, converts it to a signed type, subtracts the mid-scale offset $N_{\mathrm{off}} = 2^{15} = 32768$, and scales the result to amperes:

$$\hat{\imath}_x[k] = K_{\mathrm{dec}}\Big(N_x[k] - N_{\mathrm{off}}\Big), \qquad K_{\mathrm{dec}} = \frac{1}{K_{\mathrm{adc}} K_{cs}} = 1.53 \times 10^{-4}\,\mathrm{A}, \tag{44}$$

which is Equation (43) with K_{dec} expressed as amperes per count. The decoder gain is implemented as $V_{\mathrm{ref}}/(2^N K_{cs}) = 10/65536$, which is exactly representable in binary fixed point, therefore the decoding adds no error beyond the ADC quantization, and the mid-scale reading decodes to exactly zero current.

Encoder Decoding The `Encoder Decoder` subsystem, shown in Figure 30 , reconstructs the rotor position from the quadrature channels A and B of Equation (37). It is the inverse of the encoder emulation of Section Encoder Emulation and is built entirely from Boolean logic, comparisons, and integer additions so that it maps directly to FPGA logic. The decoder logic runs at $T_s = 5\,\text{ns}$ to match the encoder logic. The decoder operates in five stages.

- **Edge detection.** Each channel is compared with its value at the previous sample through a unit delay and an exclusive OR, producing the edge flags $e_A[k] = A[k] \oplus A[k-1]$ and $e_B[k] = B[k] \oplus B[k-1]$.

- **Direction detection.** The direction of rotation is determined by comparing the current state of channel A with the previous state of channel B, $d_f[k] = A[k] \oplus B[k-1]$. For every valid transition in Table 10, $d_f = 1$ when the count is increasing and $d_f = 0$ when it is decreasing. The value of d_f is valid only at the sample where an edge occurs. Between edges $B[k-1] = B[k]$, so d_f reduces to $A \oplus B$, which alternates with each count. The direction is therefore latched at each valid count and held between counts for use by the velocity estimate.

- **Count detection.** A valid count occurs when exactly one channel changes, $t[k] = e_A[k] \oplus e_B[k]$. When both channels change in the same sample, $e_A[k] \wedge e_B[k]$ flags an illegal transition, which corresponds to the encoder speed limit of Equation (39) being exceeded.

- **Position accumulation.** On each valid count, the position count is incremented when $d_f = 1$ and decremented otherwise. A preset input overwrites the count with a supplied value, this allows for direct overrides for debugging on the FPGA.

- **Electrical count.** A second counter tracks the position within one electrical revolution. It increments or decrements on each valid count in the same way, however

rolls over at 399 and 0. A zeroing input resets the count to zero, to allow for electrical angle calibration to be implemented.

Figure placeholder: `figures/ch2_encoder_decoder`

Alt text: Screenshot of the Encoder Decoder subsystem showing edge detection on channels A and B, direction detection and latching, valid-count detection, the position and electrical counters with rollover logic, and the velocity calculation from the time between counts.

Figure 30: The `Encoder Decoder` subsystem.

Combining these operations, the position count $\hat{n}$ and electrical count $\hat{n}_e$ are updated as

$$\hat{n}[k+1] = \begin{cases} n_{\text{preset}}, & \text{preset enabled}, \\ \hat{n}[k] + t[k]\left(2d_f[k] - 1\right), & \text{otherwise}, \end{cases} \tag{45}$$

$$\hat{n}_e[k+1] = \begin{cases} 0, & \text{zeroing enabled}, \\ \left(\hat{n}_e[k] + t[k]\left(2d_f[k] - 1\right)\right) \bmod \dfrac{N_c}{N_r}, & \text{otherwise}, \end{cases} \tag{46}$$

where the modulo operation is implemented by the comparison and rollover logic rather than by a division. The mechanical and electrical angles used by the controller are then

$$\hat{\theta}_m[k] = \frac{2\pi}{N_c}\,\hat{n}[k], \qquad \hat{\theta}_e[k] = \frac{2\pi}{N_c/N_r}\,\hat{n}_e[k], \tag{47}$$

with $N_c = 20000$ counts per mechanical revolution from Equation (35) and $N_c/N_r = 400$ counts per electrical revolution. Tracking the electrical count separately keeps the electrical angle within a single revolution without a modulo operation on the full position count, and is consistent with Equation (19) because $N_r\hat{\theta}_m$ and $\hat{\theta}_e$ differ only by whole electrical revolutions when both counters are zeroed together. The radians-per-count constants are computed from a 32-bit value of π with 28 fractional bits, a relative error of 2.8×10^{-8}. Over the full travel of 142 revolutions this accumulates to less than 0.1 counts, so the conversion adds no error beyond the encoder resolution.

The electrical angle is meaningful only if $\hat{n}_e = 0$ coincides with the alignment of the rotor d-axis with phase A as this is set as the zero position for the system to start from. An electrical angle offset is applied to adjust the zero position such that phase A is always aligned with the d-axis to start. On the physical system the rotor is first aligned by energizing phase A, after which the zeroing input sets $\hat{n}_e = 0$. In the simulation the rotor starts at $\theta_m = 0$ with the d-axis already aligned with phase A, therefore both counters start at zero and no alignment step is needed. On the physical system the alignment is performed once at power-up. With the position and velocity loops disabled, the current loops command $i_d^* > 0$ and $i_q^* = 0$ at $\hat{\theta}_e = 0$, which energizes phase A and pulls the rotor to the nearest position where the d-axis is aligned with phase A. After the rotor settles, the zeroing input sets $\hat{n}_e = 0$, the preset input sets $\hat{n}$ to the desired position reference, and closed-loop control begins.

Velocity Estimation The velocity is estimated from the time between encoder counts instead of differencing the position. A counter running at the 200 MHz logic rate, $T_{\text{clk}} = 5\,\text{ns}$, is reset at every valid count, so at each count it holds the number of clock periods N_Δ since

the previous count. The speed over that interval is

$$\hat{\omega}_m = \pm \frac{K_\omega}{N_\Delta}, \qquad K_\omega = \frac{2\pi}{N_c\, T_{\mathrm{clk}}} = 62\,831.85\,\mathrm{rad\,s^{-1}}, \tag{48}$$

with the sign given by the direction latched at that count. The counter restarts at one after each reset, so $N_\Delta \geq 1$ and the division is always defined.

Between counts, the estimate is held at its last value until the elapsed time exceeds the previous count interval. After that point the true speed must be lower than the held value, so the estimate is replaced by K_ω / N, where N is the number of clock periods since the last count, and the estimate decays as $1/t$ when the rotor slows or stops. Without this decay the estimate would hold its last nonzero value whenever the rotor stopped between counts, which destabilized the velocity loop in early simulations.

The estimate is computed as a 32-bit constant divided by the 32-bit count, with the result in `SpeedLoopType`. Division is normally avoided on an FPGA. Here a single 32-bit divider replaces the alternatives of differencing the position over a fixed window, which trades resolution against delay, or estimating the velocity with an observer, which adds both latency and resources. The velocity loop samples the estimate every 400 µs, so the latency of a pipelined divider is negligible.

The resolution of the estimate depends on speed. At the maximum speed of $35.5\,\mathrm{rad\,s^{-1}}$, counts arrive every 8.85 µs, or about 1770 clock periods, therefore adjacent values of N_Δ differ by 0.06% in speed. The estimate is the average speed over the last count interval and changes only at counts, so it lags the true speed by between one half and one and a half count intervals, about 4 µs to 13 µs at full speed. At low speed the count interval grows to 314 µs at $1\,\mathrm{rad\,s^{-1}}$, which sets the velocity-loop sample period of Section Velocity Loop. Figure 31 compares the estimate with the true rotor speed near $9\,\mathrm{rad\,s^{-1}}$.

Figure placeholder: `figures/ch2_velocity_estimate_zoom`

Alt text: Plot of estimated and true rotor speed versus time near 9 radians per second: the estimate is a staircase that updates at each encoder count and trails the smooth true speed by about one count interval.

Figure 31: Period-based velocity estimate compared with the true rotor speed.

Rotor-Frame Currents The decoded phase currents are transformed into the rotor frame by the Park transform of Equation (20), evaluated with the measured electrical angle,

$$\hat{\imath}_d = \hat{\imath}_a \cos\hat{\theta}_e + \hat{\imath}_b \sin\hat{\theta}_e, \qquad \hat{\imath}_q = -\hat{\imath}_a \sin\hat{\theta}_e + \hat{\imath}_b \cos\hat{\theta}_e. \tag{49}$$

Because the motor has two phases separated by 90°, the phase currents are already orthogonal components, no Clarke transform is required before the Park rotation [25].

Plant Models for Control Design

Each ADRC is designed around a first-order model of the form

$$\dot{y} = b_0\, u + f, \tag{50}$$

where y is the controlled output, u is the control input, b_0 is the input gain, and f is the total disturbance [24]. The total disturbance collects every term in the true dynamics other than $b_0 u$, including external loads, coupling, parameter error, and unmodeled dynamics, and is estimated by an extended state observer. Only b_0 must be known in advance.

Velocity Loop Input Gain Setting $i_d = 0$ and substituting Equation (26) into Equation (23) gives the velocity dynamics with the quadrature current as the input:

$$\dot{\omega}_m = \frac{K_t}{J}\, i_q - \frac{B_m\, \omega_m + T_L}{J}. \tag{51}$$

Comparing with Equation (50), with $y = \omega_m$ and $u = i_q^*$, the input gain is the known part of the plant,

$$b_0 = \frac{N_r\, \psi_m}{J} = \frac{K_t}{J} = \frac{K_t}{J_m + J_s + J_n}. \tag{52}$$

Using $K_t = 0.377\,12\,\mathrm{N\,m\,A^{-1}}$ and $J = 6.3761 \times 10^{-5}\,\mathrm{kg\,m^2}$,

$$b_0 = \frac{0.377\,12\,\mathrm{N\,m\,A^{-1}}}{6.3761 \times 10^{-5}\,\mathrm{kg\,m^2}} = 5914.6\,\mathrm{rad\,s^{-2}\,A^{-1}}. \tag{53}$$

The damping and load terms are not included in b_0, they are instead absorbed into the total disturbance

$$f_\omega = -\frac{B_m\, \omega_m + T_L}{J} + \frac{K_t}{J}\big(i_q - i_q^*\big) + \Delta_\omega, \tag{54}$$

where the second term is the tracking error of the current loop and Δ_ω collects unmodeled effects on the physical system such as friction, detent torque, and the ballscrew efficiency. This keeps b_0 dependent only on the torque constant and the inertia, which are the best known parameters of the system.

Current Loop Input Gain Rearranging Equations (21) and (22) for the current derivatives results in

$$\frac{di_d}{dt} = \frac{1}{L}\,v_d + \underbrace{\frac{-R\,i_d + N_r\,\omega_m L\,i_q}{L}}_{f_d}, \tag{55}$$

$$\frac{di_q}{dt} = \frac{1}{L}\,v_q + \underbrace{\frac{-R\,i_q - N_r\,\omega_m L\,i_d - N_r\,\omega_m\,\psi_m}{L}}_{f_q}. \tag{56}$$

Both axes take the form of Equation (50) with the same input gain,

$$b_i = \frac{1}{L} = \frac{1}{2.7\,\mathrm{mH}} = 370.4\,\mathrm{A\,s^{-1}\,V^{-1}}. \tag{57}$$

The winding resistance, the speed-dependent cross-coupling between the axes, and the back-EMF are all contained in the total disturbances f_d and f_q. The current loops therefore do not require decoupling or back-EMF feedforward terms, which are normally required in field-oriented control and which would require accurate values of R and ψ_m, for each motor [10].

Discrete-Time First-Order ADRC

Both the velocity loop and the current loops use the same discrete-time first-order ADRC, implemented in the minimum-footprint form of Herbst and Madonski [24]. This section derives the controller from the plant model of Equation (50) and shows how the minimum-footprint form is obtained from it. In this section T_s denotes the sample period of the loop being designed, $T_i = 20\,\mu\mathrm{s}$ for the current loops and $T_\omega = 400\,\mu\mathrm{s}$ for the velocity loop. It is distinct from the 5 ns step of the plant model.

Extended State Observer and Control Law Treating the total disturbance as an additional state, $\mathbf{x} = [\, y,\ f \,]^{\mathsf{T}}$, and discretizing Equation (50) with the forward Euler method at the sample time T_s gives

$$\mathbf{x}[k+1] = \underbrace{\begin{bmatrix} 1 & T_s \\ 0 & 1 \end{bmatrix}}_{\mathbf{\Phi}} \mathbf{x}[k] + \underbrace{\begin{bmatrix} b_0 T_s \\ 0 \end{bmatrix}}_{\mathbf{\Gamma}} u[k], \qquad y[k] = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_{\mathbf{c}^{\mathsf{T}}} \mathbf{x}[k], \tag{58}$$

in which the disturbance is modeled as constant over one sample. The extended state observer is implemented as a current estimator, which corrects the prediction from the previous sample with the measurement taken at the present sample:

$$\hat{\mathbf{x}}[k] = \big(\mathbf{I} - \mathbf{l}\,\mathbf{c}^{\mathsf{T}}\big)\Big(\mathbf{\Phi}\,\hat{\mathbf{x}}[k-1] + \mathbf{\Gamma}\,u_{\mathrm{lim}}[k-1]\Big) + \mathbf{l}\,y[k], \qquad \mathbf{l} = \begin{bmatrix} l_1 \\ l_2 \end{bmatrix}. \tag{59}$$

The observer is driven by the limited control signal u_{lim}, which is the control signal actually applied to the plant after saturation, rather than the unlimited controller output. Placing both observer poles at $z_{ESO} = e^{-\omega_{ESO} T_s}$ requires

$$l_1 = 1 - z_{ESO}^2, \qquad l_2 = \frac{\big(1 - z_{ESO}\big)^2}{T_s}. \tag{60}$$

The control law cancels the estimated disturbance and applies proportional feedback on the estimated output,

$$u[k] = \frac{u_0[k] - \hat{f}[k]}{b_0}, \qquad u_0[k] = k_1\big(r[k] - \hat{y}[k]\big), \tag{61}$$

where r is the reference. When the estimates converge, substituting Equation (61) into Equation (58) reduces the plant to $y[k+1] = y[k] + T_s k_1 (r[k] - y[k])$, a first-order system with pole at $z = 1 - k_1 T_s$. Placing this pole at $z_{CL} = e^{-\omega_{CL} T_s}$ gives

$$k_1 = \frac{1 - z_{CL}}{T_s}. \tag{62}$$

The design is therefore fully specified by four quantities: the input gain b_0, the closed-loop bandwidth ω_{CL}, the observer bandwidth $\omega_{ESO} = k_{ESO}\,\omega_{CL}$, and the sample time T_s. This is the bandwidth parameterization of Gao [20], applied to the discrete poles [24].

Minimum-Footprint Implementation Eliminating the observer states from Equations (59) and (61) gives an equivalent controller in transfer-function form, which requires only two delay elements and seven multiplications:

$$U(z) = \frac{k_1}{b_0}\,R(z) - C_y(z)\,Y(z) + C_u(z)\,U_{\lim}(z), \tag{63}$$

$$C_y(z) = \frac{\beta_0 + \beta_1 z^{-1}}{1 + \alpha_1 z^{-1} + \alpha_2 z^{-2}}, \qquad C_u(z) = \frac{\gamma_0 z^{-1} + \gamma_1 z^{-2}}{1 + \alpha_1 z^{-1} + \alpha_2 z^{-2}}, \tag{64}$$

with the coefficients

$$
\begin{aligned}
\alpha_1 &= -2z_{ESO}, \qquad \alpha_2 = z_{ESO}^2, \\
\beta_0 &= \frac{z_{CL}z_{ESO}^2 - 2z_{ESO} - z_{CL} + 2}{T_s b_0}, \\
\beta_1 &= \frac{2z_{CL}z_{ESO} - 2z_{CL}z_{ESO}^2 + z_{ESO}^2 - 1}{T_s b_0}, \\
\gamma_0 &= z_{CL}z_{ESO}^2 - 2z_{ESO} + 1, \qquad \gamma_1 = z_{ESO}^2\big(1 - z_{CL}\big).
\end{aligned}
\tag{65}
$$

The coefficients are pre-computed before running the simulation by the MATLAB function `MFP_ADRC_1st_params_multirate`. Substituting Equations (60) and (62) into Equations (59) and (61) and simplifying reproduces Equations (63) through (65) exactly, which confirms that the minimum-footprint controller and the observer-based controller are the same controller. The denominator $1 + \alpha_1 z^{-1} + \alpha_2 z^{-2} = (1 - z_{ESO}z^{-1})^2$ contains the two observer poles. The coefficients β_0 and β_1 carry the units of u per unit of y, while α_i and γ_i are dimensionless.

The `MFP_ADRC_1st` subsystems, shown in Figure 32 , implement Equation (63) as the difference equations

$$
\begin{aligned}
w[k] &= x_1[k] - \beta_0\, y[k], \\
u[k] &= \frac{k_1}{b_0}\, r[k] + w[k], \\
x_1[k+1] &= x_2[k] - \beta_1\, y[k] + \gamma_0\, u_{\mathrm{lim}}[k] - \alpha_1\, w[k], \\
x_2[k+1] &= \gamma_1\, u_{\mathrm{lim}}[k] - \alpha_2\, w[k],
\end{aligned}
\tag{66}
$$

where x_1 and x_2 are the outputs of the two unit delays and w is the feedback portion of the control signal. The reference feedforward term $k_1 r/b_0$ and the feedback term w are summed and converted to the output data type. Because $y[k]$ enters $u[k]$ directly through β_0, the controller responds to a new measurement in the same sample, which is the advantage

of the current estimator over a predictive one. Because $u_{\text{lim}}[k]$ affects only the delay states, feeding the limited signal back to the controller does not create an algebraic loop.

Figure placeholder: `figures/ch2_mfp_adrc_subsystem`

Alt text: Screenshot of the MFP ADRC first-order subsystem: two unit delays hold the states x1 and x2, gain blocks apply the coefficients alpha, beta, and gamma, and the reference term k1 over b0 times r is added to the feedback term w to form the control output u.

Figure 32: The `MFP_ADRC_1st` subsystem, which implements Equation (66).

Closed-Loop Properties For the nominal plant of Equation (58) without saturation, the closed-loop transfer functions from the reference and from the total disturbance to the output are

$$\frac{Y(z)}{R(z)} = \frac{1 - z_{CL}}{z - z_{CL}}, \qquad \frac{Y(z)}{F(z)} = \frac{T_s\,(z-1)\left(z - z_{CL} z_{ESO}^2\right)}{\left(z - z_{CL}\right)\left(z - z_{ESO}\right)^2}. \tag{67}$$

The reference response is first order with bandwidth ω_{CL}, and the observer poles cancel from it entirely, so the observer bandwidth can be chosen for disturbance rejection without affecting the reference response. The disturbance response has a zero at $z = 1$, so a constant disturbance, such as a constant load torque or a constant back-EMF, is rejected completely in steady state. The characteristic polynomial, $(z - z_{CL})(z - z_{ESO})^2$, shows that the controller and observer poles can be placed independently.

The integral action responsible for the zero at $z = 1$ resides in the feedback of u_{lim} through $C_u(z)$. Solving Equation (63) for U gives a factor of $1/(1 - C_u(z))$, which has a pole at $z = 1$ only when $C_u(1) = 1$, or equivalently

$$\gamma_0 + \gamma_1 = 1 + \alpha_1 + \alpha_2 = \big(1 - z_{ESO}\big)^2. \tag{68}$$

The coefficients of Equation (65) satisfy this identity. When the control signal saturates, u_{lim} stops following u, the loop through $C_u(z)$ no longer integrates, and the controller is protected from integrator windup without any additional anti-windup logic [24].

Because the observer absorbs modeling error into f, b_0 need not be exact. If the true input gain differs from b_0, the term $(b - b_0)\,u$ is estimated as part of the total disturbance. A value of b_0 larger than the true value reduces the effective loop gain, and a smaller value increases it, so b_0 is a robustness and tuning parameter rather than a quantity that must be identified precisely. For example, the measured phase inductance of Table 4 is about 19% above the datasheet value used for b_i, so the current loops on the physical motor run with an input gain about 16% below b_i. This lowers the effective bandwidth slightly but does not destabilize the controllers.

Position Loop

The `Position` subsystem implements a proportional controller on the mechanical position,

$$\omega_m^*[k] = \text{sat}_{\omega_{\max}}\Big(K_{P\theta}\big(\theta_m^*[k] - \hat{\theta}_m[k]\big)\Big), \tag{69}$$

where θ_m^* is the position target and the velocity reference is limited to $\pm\omega_{\max}$, which can be changed during run time, allowing for a dynamic $\omega_{\max}$ based on current system parameters. Because the velocity loop is much faster than the position loop, it can be approximated as a unity gain, and the position loop reduces to $\dot{\theta}_m = K_{P\theta}(\theta_m^* - \theta_m)$, a first-order response with

bandwidth $K_{P\theta} = 90\,\mathrm{rad\,s^{-1}}$. The time constant is $1/K_{P\theta} = 11.1\,\mathrm{ms}$, so an unsaturated step settles to within 2% in about 44 ms. A proportional controller is sufficient for the position loop because the velocity loop provides the integrating plant, the position is the integral of the regulated velocity, so a constant position error cannot persist. Load torque on the axis is rejected by the velocity ADRC before it can produce a steady-state position error.

Velocity Loop

The velocity loop applies the controller of Section Discrete-Time First-Order ADRC with $y = \hat{\omega}_m$, $r = \omega_m^*$, and $u = i_q^*$. The input gain is b_0 from Equation (53), the closed-loop bandwidth is $\omega_{CL} = 400\,\mathrm{rad\,s^{-1}}$, and the observer bandwidth is $k_{ESO} = 4$ times the closed-loop bandwidth, or $1600\,\mathrm{rad\,s^{-1}}$.

The velocity loop executes every $T_\omega = 400\,\mu\mathrm{s}$, twenty current-loop periods. The period-based velocity estimate of Section Velocity Estimation changes only when an encoder count arrives. At a 20 µs period the loop would receive a new count every sample only above $15.7\,\mathrm{rad\,s^{-1}}$, and at lower speeds it would see repeated values followed by steps, which the observer interprets as disturbance. At 400 µs, every sample contains at least one new count above $0.79\,\mathrm{rad\,s^{-1}}$.

The bandwidths were selected from a sweep of ω_{CL} and k_{ESO} on the complete model, shown in Figures 33 and 34. With $k_{ESO} = 2$ the observer is too close to the closed-loop bandwidth and the response overshoots. As $\omega_{ESO}T_\omega$ approaches one, the observer becomes too fast for the sample period combined with the delays of the current loop and the velocity estimate, and the response rings. A closed-loop bandwidth of $400\,\mathrm{rad\,s^{-1}}$ with $k_{ESO} = 4$, giving $\omega_{CL}T_\omega = 0.16$ and $\omega_{ESO}T_\omega = 0.64$, gave the smoothest response, consistent with the guidance that the observer bandwidth be several times the closed-loop bandwidth [20, 24]. The resulting coefficients are listed in Table 16.

The output of the velocity ADRC is limited to $\pm I_{pk}$ before it is used as the current

Figure placeholder: `figures/ch2_velocity_tuning_map`

Alt text: Heat map of velocity tracking error over closed-loop bandwidth and observer bandwidth multiplier, with dashed contours of constant observer bandwidth times sample period.

Figure 33: Velocity-loop tuning map: velocity tracking error over ω_{CL} and k_{ESO}, with contours of constant $\omega_{ESO} T_{\omega}$.

Figure placeholder: `figures/ch2_velocity_tuning_traces`

Alt text: Two stacked plots comparing the original tuning, 750 radians per second with observer multiplier 2, against the selected tuning, 400 radians per second with multiplier 4: position versus time on top, and actual and commanded velocity versus time below.

Figure 34: Position and velocity response with the original and selected velocity-loop tunings.

command, and the limited command is fed back to the controller as $u_{\lim}$. The limit protects the motor windings and keeps the peak phase current, equal to $|i_q|$ when $i_d = 0$, within the $\pm 4.6\,\mathrm{A}$ linear range of the revised current-sensing stage described in Section Current Sensing.

Current Loops

The `current error` subsystem contains two instances of the controller of Section Discrete-Time First-Order ADRC, one for each axis. The direct-axis loop uses $y = \hat{i}_d$, $r = i_d^* = 0$, and $u = v_d^*$. The quadrature-axis loop uses $y = \hat{i}_q$, $r = i_q^*$, and $u = v_q^*$. Both loops use the input gain b_i of Equation (57), a closed-loop bandwidth of $\omega_{CL} = 10\,000\,\mathrm{rad\,s^{-1}}$, and an observer bandwidth of $k_{ESO} = 2$ times the closed-loop bandwidth, or $20\,000\,\mathrm{rad\,s^{-1}}$. The two axes have identical models, and share one set of coefficients, listed in Table 16. Each controller output is saturated before leaving the subsystem, and the actual voltage applied on each axis is returned to the controller as $u_{\lim}$.

At the current-loop bandwidth, $\omega_{CL}T_i = 0.2$ and $\omega_{ESO}T_i = 0.4$, so the discrete poles $z_{CL} = 0.819$ and $z_{ESO} = 0.670$ remain well inside the unit circle and away from the origin. The $20\,\mu\mathrm{s}$ sample time is therefore fast enough for the chosen bandwidths without the discretization dominating the response.

Voltage Output and PWM Generation

The voltage commands are returned to the stationary frame by the inverse of the Park transform of Equation (49),

$$v_a^* = v_d^* \cos\hat{\theta}_e - v_q^* \sin\hat{\theta}_e, \qquad v_b^* = v_d^* \sin\hat{\theta}_e + v_q^* \cos\hat{\theta}_e, \tag{70}$$

and each phase voltage command is passed to an instance of the `PWM_Generator`

Table 16: ADRC design parameters and coefficients, computed with $T_\omega = 400\,\mu s$ (velocity) and $T_i = 20\,\mu s$ (current).

Quantity	Velocity loop	Current loops
b_0	$5914.6\,\mathrm{rad\,s^{-2}\,A^{-1}}$	$370.37\,\mathrm{A\,s^{-1}\,V^{-1}}$
ω_{CL}	$400\,\mathrm{rad\,s^{-1}}$	$10\,000\,\mathrm{rad\,s^{-1}}$
k_{ESO}	4	2
ω_{ESO}	$1600\,\mathrm{rad\,s^{-1}}$	$20\,000\,\mathrm{rad\,s^{-1}}$
z_{CL}	0.852144	0.818731
z_{ESO}	0.527292	0.670320
k_1	$369.64\,\mathrm{s^{-1}}$	$9063.5\,\mathrm{s^{-1}}$
k_1/b_0	$0.062\,496\,\mathrm{A\,s\,rad^{-1}}$	$24.4713\,\mathrm{V\,A^{-1}}$
α_1	-1.054585	-1.340640
α_2	0.278037	0.449329
β_0	$0.139\,568\,\mathrm{A\,s\,rad^{-1}}$	$28.1487\,\mathrm{V\,A^{-1}}$
β_1	$-0.125\,604\,\mathrm{A\,s\,rad^{-1}}$	$-25.4889\,\mathrm{V\,A^{-1}}$
γ_0	$1.823\,43 \times 10^{-1}$	$2.723\,93 \times 10^{-2}$
γ_1	$4.110\,95 \times 10^{-2}$	$8.144\,95 \times 10^{-2}$
l_1	0.721963	0.550671
l_2	$558.63\,\mathrm{s^{-1}}$	$5434.4\,\mathrm{s^{-1}}$

subsystem, shown in Figure 35 . The generator converts a signed voltage command into the gate and direction signals of Equation (12) through four operations.

- **Saturation.** The voltage command is limited to $\pm V_s$, the supply voltage, where $V_s = 48\,\mathrm{V}$.

- **Direction.** The direction signal, which selects the negative polarity of the H-Bridge, is set when the limited command is less than or equal to zero.

- **Duty computation.** The magnitude of the limited command is scaled and converted to an integer on-time n_{pwm}, in counts of a counter that runs N_{pwm} counts per PWM period. A Rate Transition block passes this value from the controller rate to the counter rate.

- **Comparison.** A free-running counter is compared with n_{pwm}, and the gate signal is high while the counter does not exceed the on-time.

The resulting duty ratio is $D_x = n_{\text{pwm}}/N_{\text{pwm}}$, and by Equation (13) the average voltage applied to the phase over one PWM period is

$$\bar{v}_x = V_s \frac{n_{\text{pwm}}}{N_{\text{pwm}}} \sigma_x, \qquad \sigma_x = 1 - 2d_x, \tag{71}$$

which is quantized in steps of V_s/N_{pwm}.

Figure placeholder: `figures/ch2_pwm_generator`

Alt text: Screenshot of the PWM Generator subsystem: the voltage command is saturated to plus or minus 48 volts, its sign sets the direction output, its magnitude is scaled to an integer on-time, and a free-running counter compared with the on-time produces the gate signal.

Figure 35: The `PWM_Generator` subsystem for one phase.

To give the current observers the voltage actually applied rather than the commanded voltage, each axis command v_d^* and v_q^* is also passed through an instance of the PWM generator, and the `Convert PWM to Voltage` subsystem reconstructs the applied voltage from the resulting on-time and direction using Equation (71). These reconstructed voltages are the u_{lim} inputs of the two current ADRCs. They include both the saturation and the PWM quantization, so the observers do not attribute either effect to the total disturbance. The generators apply the voltage limit to each axis independently, whereas the physical limit applies to each phase. The reconstructed voltages therefore match the applied voltages, to within the PWM quantization, whenever no phase saturates, and are approximate when a phase does.

Fixed-Point Implementation

The controller is implemented in fixed-point arithmetic so that it can be synthesized as HDL. The data types used are summarized in Table 17, where `fixdt(s,w,f)` is used to denote a word of w bits with f fractional bits, that is signed if $s = 1$. Each type was chosen to cover the range of the signals it holds with a resolution finer than the measurement that produces them. For example, `PosType` resolves 1.5×10^{-5} rad, which is 20 times finer than one encoder count, and covers $\pm 32\,768$ rad, or about 5200 revolutions.

Table 17: Fixed-point data types of the controller.

Name	Type	Range	Resolution	Used for
`ParamType`	`fixdt(1,32,20)`	± 2048	9.5×10^{-7}	Controller parameters
`PosType`	`fixdt(1,32,16)`	± 32768	1.5×10^{-5}	Position and position target
`CountType`	`fixdt(1,32,0)`	$\pm 2^{31}$	1	Encoder counts, decoded ADC readings
`SpeedLoopType`	`fixdt(1,32,16)`	± 32768	1.5×10^{-5}	Velocity estimate, velocity ADRC signals, i_q^*
`CurrLoopType`	`fixdt(1,32,20)`	± 2048	9.5×10^{-7}	Phase and d–q currents, current ADRC signals
`bitwiseOperator`	`fixdt(0,32,0)`	0 to $2^{32} - 1$	1	Encoder bit masks
`VelType`	`fixdt(1,128,64)`	$\pm 9.2 \times 10^{18}$	5.4×10^{-20}	Radians per count
`KomegaType`	`fixdt(1,32,14)`	± 131072	6.1×10^{-5}	Velocity constant K_ω

Quantizing the ADRC coefficients moves the controller and observer poles slightly from their design values. All coefficients are held in `ParamType`, with 20 fractional bits. The largest relative coefficient error is 1.1×10^{-5} in the velocity loop, in γ_1, and 1.7×10^{-5} in the current loops, in γ_0. The velocity observer double pole at $1600\,\mathrm{rad\,s^{-1}}$ splits to $1599.0\,\mathrm{rad\,s^{-1}}$

and $1601.0\,\mathrm{rad\,s^{-1}}$, and the current observer poles stay at $20\,000\,\mathrm{rad\,s^{-1}}$ to five significant figures. In both cases the closed-loop behavior is indistinguishable from the design.

The integral condition of Equation (68) is more sensitive than the pole locations, because the zero at $z = 1$ exists only if it holds exactly. With round-to-nearest quantization, the current-loop coefficients satisfy Equation (68) exactly. For the velocity loop, $\gamma_0 + \gamma_1$ falls one least significant bit below $1 + \alpha_1 + \alpha_2$, giving $C_u(1) = 0.9999957$. The integrator is then replaced by a pole very close to $z = 1$, and a constant load leaves a steady-state velocity error of about 4×10^{-6} of the error that would remain with no integral action, which is negligible. The identity can be made exact by computing $\gamma_1 = 1 + \alpha_1 + \alpha_2 - \gamma_0$ from the already-quantized coefficients.

Simulation Experiments

The experiments in this section verify in simulation that the controller in Section Control System regulates the motor as designed and that the quadrature current reference i_q^* produced by the velocity ADRC provides a deterministic relationship to the load on the axis.

Each experiment runs the complete closed-loop model, consisting of the stepper motor model in Section Stepper Motor Model, the PCB model in Section PCB Model, and the fixed-point controller, with the load applied through the ballscrew and measured by the emulated load cell in Section Load Cell Emulation.

Experimental Setup

Torque load is applied to the stepper motor in a linear ramp modeling the linear section of a spring. This linear load is reflected over the ballscrew to introduce an additional torque disturbance on the stepper motor modeled as T_L in Equation (33). Simulation run time is set to 1000 ms, unless noted in the specific results. A simulation run time of 1000 ms

was selected to provide enough time for the motor to produce enough displacement to be indicative of expected applications. Control system parameters were set at the values listed in Table 16 to provide consistency between experiments in the control system, only changing the parameters of the PCB system and applied torque load. The primary experiment was to reconstruct the estimated total load on the stepper motor, and compare the value to the true value in the stepper motor subsystem.

Table 18: Simulation test cases.

Test	Load	Command	Purpose
1	None	Position step	Tracking performance
2–4	Spring, $k_{s,1}$–$k_{s,3}$	Position ramp and hold	Load estimation
5	Spring and torque impulse	Position hold	Disturbance rejection

Tracking Performance

Figure 36 shows the response of the unloaded axis to a position step.

Figure placeholder: `figures/ch2_step_response`

Alt text: Three stacked plots for the unloaded position step: commanded, measured, and ideal first-order position versus time; commanded and estimated velocity versus time; and commanded quadrature current, measured quadrature current, and measured direct current versus time.

Figure 36: Response of the unloaded axis to a position step.

Table 19: Tracking performance of the unloaded axis.

Metric	Design	Simulated
Position rise time (10–90%)	24.4 ms	
Position overshoot	0%	
Steady-state position error	$< 3.14 \times 10^{-4}$ rad	
RMS i_d	0	
RMS i_q tracking error	0	

Load Estimation from the Current Reference

When the axis moves at constant velocity or is held stationary, $\dot{\omega}_m = 0$ and the current loop tracks its reference, $i_q = i_q^*$. Equation (51) then gives

$$i_q^* = \frac{T_L + B_m\,\omega_m}{K_t}. \tag{72}$$

The velocity ADRC produces this current without any model of the load, since T_L enters only through the total disturbance of Equation (54). Because the integral action of Equation (68) drives the velocity error to zero, i_q^* settles to the current the motor physically requires, independent of the value chosen for b_0. Relating the load torque to the axial force through the ballscrew, $T_L = P_h F/(2\pi\eta)$, with the lead P_h of Equation (10) and efficiency $\eta = 0.9$, gives the force estimate [1]

$$\hat{F} = \frac{2\pi\eta}{P_h}\Big(K_t\,i_q^* - B_m\,\omega_m\Big). \tag{73}$$

During the constant-velocity ramp the damping term is constant, so the rates of change of the force and the current reference are related by

$$\frac{dF/dt}{di_q^*/dt} = \frac{2\pi\eta\,K_t}{P_h}. \tag{74}$$

Figure 37 shows i_q^* and the load cell force for each spring. Figure 38 compares the

Figure placeholder: `figures/ch2_iq_and_force_vs_time`

Alt text: Plots for each spring constant of the quadrature current reference on the left axis and the load cell force on the right axis versus time, both rising linearly during the position ramp and holding constant during the hold.

Figure 37: Quadrature current reference i_q^* and load cell force for each spring.

estimate of Equation (73) with the load cell measurement, and Table 20 compares the measured slope ratio with Equation (74). In the minimum-footprint form the disturbance estimate $\hat{f}$ is not an explicit signal. In steady state, however, $\hat{y} = r$ and the control law of Equation (61) gives $\hat{f} = -b_0\, i_q^*$. The current reference is therefore the observer disturbance estimate scaled by $-1/b_0$, and Equation (72) is the same estimate expressed as a current.

Figure placeholder:
`figures/ch2_force_estimate_vs_measured`

Alt text: Scatter plot of the force estimated from the quadrature current reference versus the force measured by the load cell for all springs, with a linear fit and the ideal one-to-one line.

Figure 38: Force estimated from i_q^* compared with the load cell measurement for all springs.

Disturbance Rejection

Figure 39 shows the response to a torque impulse applied while the axis holds position against the spring.

Table 20: Measured and predicted slope ratios for each spring.

k_s	dF/dt	di_q^*/dt	Ratio, measured	Error vs. Eq. (74)

Figure placeholder: `figures/ch2_torque_impulse_response`

Alt text: Two stacked plots for a torque impulse applied while holding position against the spring: deviation of the measured position from the target versus time, and the quadrature current reference together with the applied disturbance torque divided by the torque constant versus time.

Figure 39: Response to a torque impulse while holding position against the spring.

TITLE OF MANUSCRIPT YOU ARE INCLUDING AS A CHAPTER

Contribution of Authors and Co-Authors

Manuscript in Chapter 3

Author: type author name here

Contributions: list contributions here, single-spaced

Co-Author: type author name here

Contributions: list contributions here, single-spaced

Co-Author: type author name here

Contributions: list contributions here, single-spaced

Manuscript Information

Type Author and Co-author(s) Names Here

Type name of journal here

Status of Manuscript:
____ Prepared for submission to a peer-reviewed journal
____ Officially submitted to a peer-reviewed journal
_X_ Accepted by a peer-reviewed journal
____ Published in a peer-reviewed journal

Type name of publisher here
Type date of submission here
Type date the manuscript will appear here
Type issue in which manuscript appears here
Type DOI, if available

CONCLUSION

Through the use of existing systems, data can be collected that shows a correlation between machine axes power and current cutting conditions [14]. Prior work shows the limitations of the existing systems to resolve the data collected to current cutting conditions. Through the use of a high speed system that can monitor per motor disturbances, meaningful disturbances can be measured. A low-level hardware centric approach provides a direct relationship between the current required to move the axes and the load on the axes. This relationship sets the foundation from which future work on separating the no-load motor parameters from machining disturbances can be built.

The present work concludes with a functional simulation of the desired control system and machine axis in a manner that is indicative of a physical system. Through this, the quadrature current in the stepper motor is related mathematically to the torque production. To achieve this modeling, a phase current based stepper motor model was designed that replicates the stepper motor on the physical system. A simulated model of the analog electronics on the physical PCB was designed following the expected signals of the actual PCB and FPGA. A complete cascaded control system was designed following the standard for machine control as shown in [1]. Through this the control system was designed to achieve no position overshoot, maintain steady state position and velocity as shown in Table 19, while providing a minimum quadrature current solution to the required torque of the motor.

Through the present work, the quadrature current reference was determined to provide the most direct relationship to the torque on the motor. This signal, the output of the velocity ADRC, has a higher signal-to-noise ratio making it an ideal choice of reference compared to the feedback quadrature current. The Active Disturbance Rejection Control system is able to handle anticipated disturbance loads and properly respond providing a reliable system to control a CNC machine axis.

The present work omits modeling some aspects of the PCB and stepper motor. These omissions include H-Bridge ringing, propagation delay, and latency along with stepper motor reluctance and detent torque. These effects are assumed to be small relative to the loads of interest, as stated in Section 2, and the actual FPGA implementation of the ADRC system is expected to absorb these disturbances. The sensors are modeled as ideal, and the simulation models a revised current-sensing stage, the INA241A4 with a 5 mΩ shunt, rather than the as-built PCB, whose 2.5 A measurement limit is below the 4.243 A peak current of the motor.

Future work to move forward on implementing the design on the physical system includes revising the current-sensing stage to the INA241A4 and 5 mΩ shunt modeled in this work, which removes the 2.5 A clipping, and reworking the low-pass filter to allow for filtering at higher than 50 kHz without phase delay. This filtering would allow for the accurate sampling of the phase currents synchronized with PWM pulses to avoid sampling ripple current. Further work will need to implement an embedded Linux code to access the available FPGA fabric registers to control the control system and implement FPGA IO handling to connect the FPGA to the PCB. Once completed, work can begin on implementation of a high level motion profile system that can be paired with G-Code processing for integration into existing CNC Equipment.

Through the use of Field Oriented Control on two-phase hybrid stepper motors, the quadrature current can be related to the torque load on the motor. Controlling the motor position and velocity through the quadrature current provides a reference signal of the minimum current required to generate motion and overcome the torque. This signal is directly proportional to the required torque by the torque constant K_t of the stepper motor without the detent torque and with identical inductance per phase.

REFERENCES CITED

[1] Yusuf Altintas. *Manufacturing Automation: Metal Cutting Mechanics, Machine Tool Vibrations, and CNC Design*. Cambridge University Press, Cambridge, UK, 2nd edition, 2012. ISBN 978-1-107-00148-0.

[2] Analog Devices (Maxim Integrated). *MAX11162: 16-Bit, 500ksps, +5V Unipolar Input, SAR ADC, in Tiny 10-Pin μMAX*, March 2017. URL `https://www.analog.com/media/en/technical-documentation/data-sheets/MAX11162.pdf`. Datasheet, Rev. 2.

[3] Analog Devices (Trinamic). *TMC2209 Datasheet: Power Driver for Stepper Motors*, 2023. URL `https://www.analog.com/media/en/technical-documentation/data-sheets/TMC2209_datasheet_rev1.09.pdf`. Rev. 1.09.

[4] Brian Armstrong-Hélouvry, Pierre Dupont, and Carlos Canudas de Wit. A survey of models, analysis tools and compensation methods for the control of machines with friction. *Automatica*, 30(7):1083–1138, 1994. doi: 10.1016/0005-1098(94)90209-7.

[5] Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, Princeton, NJ, 2nd edition, 2021. ISBN 978-0-691-19398-4.

[6] A. Benchaib, S. Poullain, J. L. Thomas, and J. C. Alacoque. Discrete-time field-oriented control for SM-PMSM including voltage and current constraints. In *Proceedings of the IEEE International Electric Machines and Drives Conference (IEMDC'03)*, volume 2, pages 999–1005, Madison, WI, 2003. IEEE. doi: 10.1109/IEMDC.2003.1210357.

[7] Fabio Bernardi, Emilio Carfagna, Giovanni Migliazza, Giampaolo Buticchi, Fabio Immovilli, and Emilio Lorenzani. Performance analysis of current control strategies for hybrid stepper motors. *IEEE Open Journal of the Industrial Electronics Society*, 3: 460–472, 2022. doi: 10.1109/OJIES.2022.3185659.

[8] Caleb Alexander Binfet. Implementation and resource analysis of FPGA-based multiphase motor control architectures. Master's thesis, Montana State University, Bozeman, MT, May 2026.

[9] Vladimir Blasko, Vikram Kaura, and Walter Niewiadomski. Sampling of discontinuous voltage and current signals in electrical drives: a system approach. *IEEE Transactions on Industry Applications*, 34(5):1123–1130, 1998. doi: 10.1109/28.720453.

[10] Fernando Briz, Michael W. Degner, and Robert D. Lorenz. Analysis and design of current regulators using complex vectors. *IEEE Transactions on Industry Applications*, 36(3):817–825, 2000. doi: 10.1109/28.845057.

[11] Caron Engineering. TMAC – machine tool monitoring system. https://www.caroneng.com/products/tmac/, n.d. Accessed: Nov. 1, 2026.

[12] Anton Cervin, Dan Henriksson, Bo Lincoln, Johan Eker, and Karl-Erik Årzén. How does control timing affect performance? analysis and simulation of timing using Jitterbug and TrueTime. *IEEE Control Systems Magazine*, 23(3):16–30, 2003. doi: 10.1109/MCS.2003.1200240.

[13] Hyeon Sung Cho, Ji Hyeong Han, Su Young Chi, and Kwan Hee Yoo. A tool breakage detection system using load signals of spindle motors in CNC machines. In *Proceedings of the 8th International Conference on Ubiquitous and Future Networks (ICUFN)*, pages 160–163, Vienna, Austria, July 2016. IEEE. doi: 10.1109/ICUFN.2016.7537007.

[14] Samuel Civello. Something manufacturing. Master's thesis, Montana State University, Bozeman, MT, August 2026.

[15] Cutting Tool Engineering. Tool monitoring adaptive control system, TMAC version 3.0. https://ctemag.com/?p=102899, September 2020. Accessed: Nov. 1, 2026.

[16] Stijn Derammelaere, Bram Vervisch, Frederik De Belie, Johannes Cottyn, Griet Van den Abeele, Peter Cox, Kurt Stockman, and Lieven Vandevelde. A linear time-invariant model for a vector-controlled two-phase stepping motor. In *Proceedings of the 5th IET International Conference on Power Electronics, Machines and Drives (PEMD 2010)*, pages 1–5, Brighton, UK, April 2010. IET. doi: 10.1049/cp.2010.0149.

[17] DINGS' Motion USA. EK2 encoder – rotary stepper NEMA 23 motor options. https://www.dingsmotionusa.com/rotary-stepper-nema-23, 2026. Incremental encoder option for size 23 stepper motors, single-ended and differential output. Accessed: Nov. 1, 2026.

[18] DINGS' Motion USA. Rotary stepper NEMA 23 motors – standard & custom. https://www.dingsmotionusa.com/rotary-stepper-nema-23, 2026. Specifications for model 23H2065. Accessed: Nov. 1, 2026.

[19] Gene F. Franklin, J. David Powell, and Michael L. Workman. *Digital Control of Dynamic Systems*. Addison-Wesley, Menlo Park, CA, 3rd edition, 1998. ISBN 978-0-201-82054-6.

[20] Zhiqiang Gao. Scaling and bandwidth-parameterization based controller tuning. In *Proceedings of the 2003 American Control Conference*, volume 6, pages 4989–4996, Denver, CO, 2003. IEEE. doi: 10.1109/ACC.2003.1242516.

[21] Mikell P. Groover. *Fundamentals of Modern Manufacturing: Materials, Processes, and Systems*. John Wiley & Sons, Hoboken, NJ, 4th edition, 2010. ISBN 978-0-470-46700-8.

[22] B. Henke, O. Sawodny, S. Schmidt, and R. Neumann. Modeling of hybrid stepper motors for closed loop operation. *IFAC Proceedings Volumes*, 46(5):177–183, 2013. doi: 10.3182/20130410-3-CN-2034.00042. 6th IFAC Symposium on Mechatronic Systems, Hangzhou, China.

[23] Gernot Herbst. A minimum-footprint implementation of discrete-time ADRC. In *Proceedings of the 2021 European Control Conference (ECC)*, pages 107–112, Delft, Netherlands, 2021. IEEE. doi: 10.23919/ECC54610.2021.9655120.

[24] Gernot Herbst and Rafal Madonski. Tuning and implementation variants of discrete-time ADRC. *Control Theory and Technology*, 21(1):72–88, 2023. doi: 10.1007/s11768-023-00127-0.

[25] Chiu-Keng Lai, Jhang-Shan Ciou, and Chia-Che Tsai. The modelling, simulation and FPGA-based implementation for stepper motor wide range speed closed-loop drive system design. *Machines*, 6(4):56, November 2018. doi: 10.3390/machines6040056.

[26] Toshimichi Moriwaki. Multi-functional machine tool. *CIRP Annals – Manufacturing Technology*, 57(2):736–749, 2008. doi: 10.1016/j.cirp.2008.09.004.

[27] Sohan Nagaraj and Nancy Diaz-Elsayed. Tool condition monitoring in the milling of low- to high-yield-strength materials. *Machines*, 13(4):276, March 2025. doi: 10.3390/machines13040276.

[28] Christoffer Nilsson and Daniel Modrack. Universal embedded motor control. Master's thesis, Chalmers University of Technology, Göteborg, Sweden, September 2013.

[29] Erik Oberg, Franklin D. Jones, Holbrook L. Horton, Henry H. Ryffel, and Christopher J. McCauley. *Machinery's Handbook*. Industrial Press, South Norwalk, CT, 31st edition, 2020. Edited by Laura Brengelman.

[30] Okuma America Corporation. Partners in technology: Caron Engineering. `https://www.okuma.com/partners/caron-engineering`, n.d. Accessed: Nov. 1, 2026.

[31] J. Francis Reintjes. *Numerical Control: Making a New Technology*. Oxford University Press, New York, NY, 1991. ISBN 978-0-19-506772-9.

[32] Stefano Ricci and Valentino Meacci. Simple torque control method for hybrid stepper motors implemented in FPGA. *Electronics*, 7(10):242, October 2018. doi: 10.3390/electronics7100242.

[33] Xuexiang Sheng, Zhanchen Zhu, and Changlin Liu. A review of the machining mechanisms in field-assisted cutting of brittle materials. *Micromachines*, 17(3):361, March 2026. doi: 10.3390/mi17030361.

[34] Fei Tao, Qinglin Qi, Ang Liu, and Andrew Kusiak. Data-driven smart manufacturing. *Journal of Manufacturing Systems*, 48:157–169, 2018. doi: 10.1016/j.jmsy.2018.01.006.

[35] Roberto Teti, Krzysztof Jemielniak, Garret O'Donnell, and David Dornfeld. Advanced monitoring of machining operations. *CIRP Annals – Manufacturing Technology*, 59(2): 717–739, 2010. doi: 10.1016/j.cirp.2010.05.010.

[36] Texas Instruments. *INA241x –5 V to 110 V, Bidirectional, Ultra-Precise Current Sense Amplifier With Enhanced PWM Rejection*. Dallas, TX, October 2023. URL `https://www.ti.com/lit/ds/symlink/ina241a.pdf`. Datasheet SBOSA30C, Mar. 2022, revised Oct. 2023.

[37] THK Co., Ltd. *Ball Screw General Catalog (MDK/MBF/BIF/BNF)*. Tokyo, Japan, n.d. Model BIF precision ball screw, unfinished shaft ends. Accessed: Nov. 1, 2026.

[38] Voron Design. Configuring sensorless homing. `https://docs.vorondesign.com/tuning/sensorless.html`, n.d. Voron Documentation. Accessed: Nov. 1, 2026.

[39] Shi Yuen Wong, Joon Huang Chuah, and Hwa Jen Yap. Technical data-driven tool condition monitoring challenges for CNC milling: a review. *The International Journal of Advanced Manufacturing Technology*, 107(11):4837–4857, April 2020. doi: 10.1007/s00170-020-05303-z.

[40] Randall Brock Woodrum. Developing lathing parameters for PBX 9501. Master's thesis, Texas Tech University, Lubbock, TX, August 2017. URL `https://www.osti.gov/servlets/purl/1410981`.

APPENDIX

EXAMPLE CODE

This work used the following MATLAB code to produce basically nothing. But, I'll include it here for the sake of an example.

```
% MATLAB code to say 'hello world'
disp('Hello world')
```